System Verification and Validation Plan for Flexin'

Team 3, Flexin'
Arjun Karthik
Shao (Daniel) Hu
Noah Goldschmied
Peter Deforest
Daniel Franze-Da Silva

November 2, 2026

Revision History

Date	Version	Notes
November 2, 2026	1.0	Revision 0, unit testing section omitted from this version

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 2
- 2.3 Custom Documents . . . 3
- 2.4 Relevant Documentation . . . 3

3 Plan **4**
- 3.1 Verification and Validation Team . . . 4
- 3.2 SRS Verification . . . 6
- 3.3 Design Verification . . . 6
- 3.4 Verification and Validation Plan Verification . . . 7
- 3.5 Implementation Verification . . . 7
- 3.6 Automated Testing and Verification Tools . . . 8
- 3.7 Software Validation . . . 9

4 System Tests **9**
- 4.1 Tests for Functional Requirements . . . 9
  - 4.1.1 Account Management . . . 10
  - 4.1.2 Fitness Tracking and Goals . . . 15
  - 4.1.3 Rewards and Virtual Currency . . . 18
  - 4.1.4 Simulated Investing . . . 20
  - 4.1.5 Community and Leaderboards . . . 23
  - 4.1.6 Progress Reporting . . . 25
- 4.2 Tests for Nonfunctional Requirements . . . 26
  - 4.2.1 Look and Feel . . . 26
  - 4.2.2 Usability and Humanity . . . 36
  - 4.2.3 Performance . . . 42
  - 4.2.4 Operational and Environmental . . . 56
  - 4.2.5 Maintainability and Support . . . 58
  - 4.2.6 Security . . . 62
  - 4.2.7 Cultural . . . 75
  - 4.2.8 Compliance . . . 78
  - 4.2.9 User Documentation and Training . . . 83
- 4.3 Traceability Between Test Cases and Requirements . . . 87

5 Unit Test Description **94**
5.1 Unit Testing Scope . . . 95
5.2 Tests for Functional Requirements . . . 95
5.3 Tests for Nonfunctional Requirements . . . 95
5.4 Traceability Between Test Cases and Modules . . . 95

6 Appendix **96**
6.1 Symbolic Parameters . . . 96
6.2 Usability Survey Questions . . . 98

7 Appendix — Generative AI Use Disclosure **100**
7.1 AI Tools Used . . . 100
7.2 How and Where Used . . . 100
7.3 Verification of AI-Generated Content . . . 100

8 Appendix — Reflection **102**

List of Tables

1 Abbreviations and acronyms . . . iv
2 Verification and Validation Team Roles . . . 5
3 Requirements covered: Account Management . . . 10
4 Requirements covered: Fitness Tracking and Goals . . . 15
5 Requirements covered: Rewards and Virtual Currency . . . 18
6 Requirements covered: Simulated Investing . . . 20
7 Requirements covered: Community and Leaderboards . . . 23
8 Requirements covered: Progress Reporting . . . 25
9 Requirements covered: Appearance . . . 27
10 Requirements covered: Style . . . 32
11 Requirements covered: Productization . . . 57
12 Traceability between system tests and requirements . . . 87
13 Symbolic constants . . . 96

1 Symbols, Abbreviations, and Acronyms

Table 1 lists the abbreviations and acronyms used in this document. Requirement IDs (for example FUNC-01, PR-SL-01, and SEC-ACC-01) use the prefixes defined in the ID prefixes table of the SRS (**?**). Each system test is named `test-` followed by the ID of the requirement it verifies (for example test-PR-SL-01). When one requirement has more than one test, a two-digit number is added to the end (for example test-FUNC-01-01 and test-FUNC-01-02). Names written in sans serif are symbolic constants, and their values are given in Table 13.

Table 1: Abbreviations and acronyms

Abbreviation	**Meaning**
AES	Advanced Encryption Standard
AI	Artificial intelligence
API	Application programming interface
CD	Custom document
CI/CD	Continuous integration and continuous delivery
CVE	Common Vulnerabilities and Exposures
DAST	Dynamic application security testing
dp	Density-independent pixel (Android unit of screen size)
ETF	Exchange-traded fund
FLX	Virtual currency symbol used in Flexin'
fps	Frames per second
GCM	Galois/Counter Mode
HTTP	Hypertext Transfer Protocol
ID	Identifier
ISO/IEC	International Organization for Standardization and International Electrotechnical Commission
JSON	JavaScript Object Notation
MFA	Multi-factor authentication
MIS	Module Interface Specification
OTP	One-time password
PDF	Portable Document Format
PII	Personally identifiable information
QA	Quality assurance
Rev 0	Revision 0 demonstration
SAST	Static application security testing

Abbreviation	**Meaning**
SDK	Software development kit
SHA	Secure Hash Algorithm
SQL	Structured Query Language
SRS	Software Requirements Specification
TA	Teaching assistant
TLS	Transport Layer Security
UI	User interface
UTC	Coordinated Universal Time
UX	User experience
VnV	Verification and validation
WCAG	Web Content Accessibility Guidelines
XSS	Cross-site scripting

This document describes the Verification and Validation (VnV) plan for Flexin', a fitness and finance application that allows users to learn about the power of compounding through working out and investing. This plan describes the verification and validation activities that will be performed to ensure that the software meets its requirements and is fit for its intended purpose. Section 2 summarizes the software and the objectives of the plan. Section 3 describes the team's roles and how the SRS, design, implementation, and this plan will be verified. Section 4 lists the system tests for the functional and nonfunctional requirements and traces them to the SRS. Section 5 will describe the unit tests once the detailed design is complete. The appendices give the symbolic constants, the usability survey questions, the generative AI use disclosure, and the team's reflection.

2 General Information

This section provides an overview of the software being tested, the objectives of the VnV plan, and the relevant documentation that will be used to support the VnV activities.

2.1 Summary

The software being tested is Flexin', an app that allows users to learn about the power of compounding through working out and investing. The team will be looking to test the following functions of the app:

- Step tracking and integrations with fitness data sources (Android Health Connect and, optionally, Garmin watches)
- Real-time updates of fitness data within the app
- Converting fitness progress into virtual currency and depositing it into the user's account
- Viewing account balances
- Simulated/generated financial markets
- Performance leaderboards

2.2 Objectives

The main objective of the VnV plan is to ensure that Flexin' properly teaches the impact of compounding. For the VnV plan to achieve this objective, the team will prioritize the following qualities:

- **Learnability:** The app is easy for new users to learn, as measured by the learnability requirement in the SRS (UHR-EOU-08).
- **Correctness of currency calculations:** Virtual currency earned, deposited, and displayed in account balances is calculated correctly, and operations that change balances either complete fully or are rolled back (PR-RFT-02).
- **Consistency of asset calculations:** Simulated asset values are consistent and produce the returns expected from the implemented market algorithms, including the calibration against historical market data required by PR-PA-06.
- **User confidence in the financial lessons:** Users come away trusting and understanding the financial lessons the app is intended to convey.

The following objectives are out of scope for this VnV plan:

- **Assumptions in the SRS and Hazard Analysis:** The assumptions listed in the SRS (**?**) and the Critical Assumptions section of the Hazard Analysis are taken as given and will not be verified.
- **Third-party packages and libraries:** External dependencies, such as Android Health Connect and the Garmin Connect IQ SDK, are assumed to be correct and already verified by their implementation teams (e.g. Garmin). The team does not have the time or resources to verify these libraries within the scope of this capstone. This matches the Hazard Analysis assumption that third-party health platforms keep their documented behaviour. Flexin''s own handling of outages and malformed data from these sources is still in scope (PR-RFT-03).
- **Internet connectivity:** The SRS lists a mobile data or Wi-Fi connection as part of the environment needed to reach the Flexin' backend. The strength and reliability of the user's connection will not be tested. The app's required offline behaviour (PR-RFT-01 and PR-RFT-04) is still in scope.

- **Full verification of user-reported fitness activity:** The team will not attempt to fully verify that users are not lying about completing fitness goals, as this would require too much time and resources. The SRS already assumes that fitness devices report correct data and that artificial changes are dishonest use, and SC-04 limits this risk by not allowing manual entry of activity. Small checks may be used to gauge validity in some cases (e.g. checking that a Garmin watch's heart rate reading stays above a threshold for a period of time), but validity cannot be guaranteed, especially for users without a Garmin device who track activity solely through their phone.

2.3 Custom Documents

The team is completing two custom documents (CDs), one per term:

- **CD-Fall — User/Stakeholder/Client Interview Report:** A report on interviews with the project's client and potential users, used to elicit requirements. The SRS appendix already summarizes the two interviews conducted so far: a mock interview with the client and supervisor, and an interview with a representative end user.
- **CD-Winter — Business Plan:** A business plan exploring how Flexin' could be monetized after the capstone, as the team believes the project has the potential to become a startup. It will cover the market need, market size and business model, costs, competition, and an execution plan.

2.4 Relevant Documentation

The following documents are relevant to the VnV efforts:

- **Software Requirements Specification (SRS)** (**?**): Defines the functional and non-functional requirements of Flexin', each with a fit criterion. These fit criteria are what the tests in this plan check against. The SRS also lists the assumptions that bound what is in scope for verification, and names the testing tools the team will use (JUnit for backend unit and system tests, and Detox for end-to-end tests of the mobile client).

- **Hazard Analysis:** Identifies the hazards of using Flexin' and defines twelve safety and security requirements (SR1–SR12) to mitigate them, each with a fit criterion. Its roadmap commits to implementing SR1, SR2, SR3, SR6, SR8, SR9, SR11, and SR12 within the capstone, so these are the safety and security requirements this plan verifies. Its Critical Assumptions are excluded from verification.
- **Health Connect and Garmin Connect IQ SDK documentation:** Describes how activity data is provided to Flexin'. Since these platforms are assumed to keep their documented behaviour, their documentation defines the data the integration is expected to receive.
- **Custom Documents:** The interview report (CD-Fall) records the user and client needs that the requirements were derived from, which is what validation testing checks the app against. The business plan (CD-Winter) has little direct bearing on VnV, but one SRS scalability requirement refers to it for how the product might grow after the capstone.
- **User interviews and user testing:** The interviews in the SRS appendix ground the requirements in real user needs. The SRS also plans user testing on demo builds in Term 2 (Weeks 17 and 21), which will be used to assess learnability and user confidence in the financial lessons.

3 Plan

This section outlines the plan to verify and validate Flexin' across its project development lifecycle. The plan will detail the roles and responsibilities of the team, and will specify the approach to verify the SRS document, design and implementation, as well as how to verify the VnV itself. Lastly, it will describe the automated tools and software validation framework that the team plans on using.

3.1 Verification and Validation Team

The process of VnV will be a collective effort spread across the team and supervisor. All members will have defined roles that altogether will cover the process. These roles can be seen in Table 2.

Table 2: Verification and Validation Team Roles

Team Member	**Project Role**	**VnV Responsibilities**
Peter Deforest	Testing Lead	Coordinating all VnV activities, ensuring unit, integration and regression tests are written and maintained, and performing the final review of all test cases before each milestone.
Arjun Karthik	GitHub Manager, CI/CD Lead	Implementing and maintaining the CI/CD pipeline and static analysis tools, monitoring automated test coverage metrics, and resolving issues with VnV tooling.
Shao (Daniel) Hu	Lead Developer	Leading code inspections using a defined checklist, verifying that the design and implementation satisfy the SRS, and leading the code walkthrough for the supervisor.
Daniel Franze-Da Silva	Scribe, UI/UX Design Lead	Documenting VnV meeting minutes, preparing and running the usability survey, and verifying that the UI and UX meet the usability requirements in the SRS.
Noah Goldschmied	Team Liaison, Chairman	Maintaining traceability between requirements and test cases, coordinating SRS and VnV Plan reviews, and ensuring feedback from the supervisor, TAs and peer reviewers is addressed through GitHub issues.
Chris Schankula	Project Supervisor	Providing technical oversight and domain feedback on the fitness and finance features, reviewing the SRS and VnV Plan, and participating in the Revision 0 demonstration to validate the requirements.

3.2 SRS Verification

The SRS will be verified to ensure that all requirements detailed in the document are reasonable, accurate, understandable and complete. The bullet list below describes the different avenues in which the SRS may be reviewed.

- **Submission to TA:** The SRS will be submitted to our course TA, who will thoroughly read and analyze the document and provide feedback where needed.
- **Peer Review:** Classmates will review the SRS document and ensure its content makes sense from a course standpoint.
- **Presentation to Supervisor:** The team may meet with the project supervisor to ensure that the requirements described in the SRS are high quality and in line with what they expect to be delivered.

If any problems are found with the SRS, the team will utilize our issue board to keep track and fix them. Once development begins, these issues may also coincide with a bugfix.

3.3 Design Verification

Design verification will begin after the submission of the Design Document on November 16th. This verification will ensure that Flexin's architecture meets requirements and is implementable. The bullet list below describes how the Design Document may be reviewed.

- **Submission to TA:** The Design Document will be submitted to our course TA, who will thoroughly read and analyze the document and provide feedback where needed.
- **Peer Review:** Classmates will review the Design document and ensure its content makes sense from a course standpoint.
- **Presentation to Supervisor:** The team may meet with the project supervisor to ensure that the design is feasible from an implementation point of view.

Any problems with the Design Document will be handled through the team's issue board.

3.4 Verification and Validation Plan Verification

VnV plan verification will take place before the document is submitted by the team, as well as post submission by other members.

- **Team Review:** The team will review the VnV Plan ensuring that the defined tasks are feasible within the timeframe of the course and that the detail described in the document is appropriate.
- **Peer Review:** Classmates will review the VnV Plan and ensure its content makes sense from a course standpoint.
- **Presentation to Supervisor:** The team may meet with the project supervisor to ensure that the VnV Plan is feasible.
- **Submission to TA:** The VnV Plan will be submitted to the TA for grading and feedback.

3.5 Implementation Verification

Implementation verification ensures that the system, as built, correctly implements the SRS and design document. This will be done through a combination of the following:

- **Unit Testing and System Testing:** All functional and non-functional requirements will be verified through the tests described in the below sections of this document.
- **Static Analysis and Linting:** The backend follows the Sun/Oracle Java code conventions, checked by Checkstyle using the project's `checkstyle.xml`, which is adapted from Checkstyle's `sun_checks.xml`. Checkstyle runs in the Maven `verify` phase and fails the build on any warning. The mobile app follows the Expo ESLint configuration (`eslint-config-expo`) with Prettier formatting, and is type-checked with the TypeScript compiler (`tsc --noEmit`).
- **Code Review:** All code changes will require a pull request and approval from at least one team member. The team members will be looking for code correctness, standards, and proper error handling.

3.6 Automated Testing and Verification Tools

Flexin' has two codebases, and each has its own tools:

- **Backend (`src/api`):** Written in Java 21 with Spring Boot and built with Maven. Unit and integration tests use JUnit 5 with Mockito, through the Spring Boot test starters. In CI they run against a MySQL 9 test database, the same database engine used in production. Static analysis uses Checkstyle, as described in Section 3.5.

- **Mobile app (`src/mobile`):** Written in TypeScript with React Native and Expo. Unit and component tests use Jest (with the `jest-expo` preset) and React Native Testing Library. End-to-end tests use Detox on an Android emulator. Linting uses ESLint and formatting uses Prettier.

Continuous integration: GitHub Actions runs the App CI workflow on every pull request and push to `main` that changes the source code. For the mobile app it runs `npm run lint`, `npm run typecheck`, `npm run format:check`, and `npm test`. For the backend it runs `./mvnw verify`, which compiles the code, runs the JUnit tests, and runs Checkstyle. GitHub requires another team member to approve a pull request before it is merged, and the team will not merge a pull request whose App CI checks fail. Detox end-to-end tests take longer, so they are run before each milestone demo instead of on every pull request.

Code coverage: The team will measure coverage with JaCoCo for the backend, added as a Maven plugin, and with Jest's built-in coverage report (`jest --coverage`) for the mobile app. Both tools can fail the build when coverage falls below a threshold, which the team will set to CoverageTarget once the core features are implemented. Coverage reports will be uploaded as CI artifacts so reviewers can see which code a pull request leaves untested.

Performance measurement: Timing tests (Section 4.2.3) use screen recordings on the reference phone and timestamps logged by the Detox scripts. The team does not plan to use a memory profiler such as Valgrind, since the backend runs on the JVM and the app on React Native, where memory is managed automatically.

3.7 Software Validation

The software validation step of the VnV plan will ensure that the near-final product lives up to its expectations. This validation will be done from the perspective of the users and the stakeholders.

- **Rev-0 Demonstration:** The Rev-0 demo will be used to ensure that the core functionality of Flexin' is up to the standards of what the supervisor expects.
- **Usability Tests:** The team plans on giving Flexin' to a select group of prospective users for a period of roughly UsabilityTestPeriod. This will help the team understand what needs work, and what worked well for the users.

4 System Tests

This section describes the system tests that will be performed to verify and validate the functional and non-functional requirements of Flexin'. The tests for functional requirements come first, followed by the tests for nonfunctional requirements, both grouped in the same order as the SRS (**?**). The section ends with a table that traces each test to the requirement it verifies.

The mandated constraints in the SRS (SC-01 to SC-06) limit how Flexin' is built rather than how it behaves, so they do not have system tests of their own. SC-02 and SC-03, which require a TypeScript and React Native client and a Java backend, are checked by code review and by the CI build described in Section 3.6. The other constraints are checked as part of existing tests: SC-01 (Android) by test-PR-L-01, SC-04 (automatic activity data) by test-FUNC-03, SC-05 (simulated market only) by test-PR-SC-03 and test-LF-STY-02, and SC-06 (app store distribution) by test-PROD-02. The traceability table lists these under "Also Supports".

4.1 Tests for Functional Requirements

The subsections below group the functional requirements in the Functional Requirements section of the SRS (**?**) by feature area. Together they cover FUNC-01 to FUNC-16. FUNC-17 (Sponsored Reward Redemption) is a stretch goal in the SRS Waiting Room, so it is not tested.

Each feature area starts with a table of the requirements it covers, followed by one table per test case. Each test case gives its control (manual or automatic), initial state, inputs, expected output, the requirement it covers, and how the test will be performed.

4.1.1 Account Management

These tests cover creating an account and signing in.

Table 3: Requirements covered: Account Management

SRS ID	**Requirement**
FUNC-01	User Registration
FUNC-02	User Authentication

test-FUNC-01-01	**User Registration - Account Does Not Already Exist**
Control	Manual
Initial State	User is not logged in and does not possess an existing Flexin' account.
Inputs	A valid email address, a valid password, and a call to the create account action.
Output	A new user account is created successfully and the user is granted access to the application using the newly created credentials.
Test Case Derivation	This test covers FUNC-01. Users can create an account using a valid email address and password.
How test will be performed	On the registration screen a valid email (that is not already associated with an existing account) and custom password will be entered, the form will be submitted verifying that the account creation was successful. Then, on the login screen, an attempt to log in with the new credentials will be made verifying access to the application under this newly created account.

test-FUNC-01-02	**User Registration - Account Already Exists**
Control	Manual
Initial State	User is not logged in and already possesses a Flexin' account.
Inputs	A valid email address (already associated with an existing account), a valid password, and a call to the create account action.
Output	The system rejects the request to create a new account and prompts the user to sign in instead.
Test Case Derivation	This test covers FUNC-01. Users cannot create multiple accounts with the same email address.
How test will be performed	On the registration screen a valid email (that is already associated with an existing account) and custom password will be entered, the form will be submitted verifying that the account creation was unsuccessful. Then, the system will prompt the user to sign in.

test-FUNC-02-01	**User Authentication - Correct Credentials**
Control	Manual
Initial State	User is not logged in and already possesses a Flexin' account.
Inputs	A valid email address (already associated with an existing account), a password that matches the one associated with the account, and a call to the login action.
Output	The user is successfully authenticated and the system grants access to the application using the provided credentials.
Test Case Derivation	This test covers FUNC-02. Users are authenticated before accessing personalized application functionality.
How test will be performed	On the login screen, a valid email (already associated with an existing account) and the corresponding password will be entered, and the form will be submitted verifying that access to the application was granted. Any account information associated to the authenticated user will be available for viewing and editing.

test-FUNC-02-02	**User Authentication - Incorrect Password**
Control	Manual
Initial State	User is not logged in and already possesses a Flexin' account.
Inputs	A valid email address (already associated with an existing account), an incorrect password (one that does not match the one associated with the account), and a call to the login action.
Output	The user is denied access and the system displays an error message indicating invalid credentials were entered.
Test Case Derivation	This test covers FUNC-02. Users are denied access to the application and to account-specific information with incorrect credentials.
How test will be performed	On the login screen, a valid email (already associated with an existing account) and a password that does not match the one associated with the email will be entered, and the form will be submitted verifying that access to the application was denied.

test-FUNC-02-03	**User Authentication - Incorrect Email**
Control	Manual
Initial State	User is not logged in and already possesses a Flexin' account.
Inputs	An invalid email address (not associated with any existing account).
Output	The user is denied access and the system displays an error message indicating that an account with the entered email address does not exist.
Test Case Derivation	This test covers FUNC-02. Users are denied access to the application without existing and valid credentials.
How test will be performed	On the login screen, an invalid email (not associated with any existing account) will be entered, and the form will be submitted verifying that access to the application was denied. Then, the system will prompt the user to create a new account with the entered email address.

test-FUNC-02-04	**User Authentication - Reset Password**
Control	Manual
Initial State	User is not logged in, already possesses a Flexin' account, and has forgotten their password.
Inputs	A valid email address (already associated with an existing account) and a call to the reset password action.
Output	The system sends a password reset link to the user's email address. The user can then use this link to successfully set a new password and login.
Test Case Derivation	This test covers FUNC-02. Users can reset their password and gain access to their account once the reset process is completed.
How test will be performed	On the login screen, a valid email (already associated with an existing account) will be entered and the reset password action will be triggered, and then instructions will be followed to complete the password reset process. Upon completion, an attempt to log in with the new password will be made verifying that access to the application was granted.

4.1.2 Fitness Tracking and Goals

These tests cover importing activity data from third-party providers and creating and tracking fitness goals.

Table 4: Requirements covered: Fitness Tracking and Goals

SRS ID	**Requirement**
FUNC-03	Third-Party Fitness Data Import
FUNC-04	Fitness Goal Creation
FUNC-05	Fitness Goal Tracking

test-FUNC-03	Third-Party Fitness Data Import
Control	Automatic
Initial State	User already possesses a Flexin' account, and has granted permission for their data to be imported to the Flexin' application from their desired and approved third-party provider.
Inputs	Sync a fitness activity that consists of 1,000 or more recorded steps.
Output	The imported fitness activity appears in the user's activity history after synchronization.
Test Case Derivation	This test covers FUNC-03. The system successfully imports fitness data from third-party providers.
How test will be performed	A Garmin will be connected to the Flexin' application, 1,000 steps will be walked through the day and will sync the 1,000 or more recorded steps. After syncing, the system will import the data into the user's activity history verifying that activity details match the original source.

test-FUNC-04-01	Fitness Goal Creation - Create Goal
Control	Manual
Initial State	User is logged into their account.
Inputs	An fitness activity with 1,000 or more recorded steps is entered as a fitness goal.
Output	The fitness goal is successfully created and displayed in the user's goal list.
Test Case Derivation	This test covers FUNC-04. The system successfully allows users to create fitness goals.
How test will be performed	The user will navigate to the fitness goal creation screen. They will enter a fitness activity with 1,000 or more recorded steps and submit the form to register this fitness goal. Then it will be verified that the system correctly displays the new fitness goal in the user's goal list.

test-FUNC-04-02	**Fitness Goal Creation - Edit Goal**
Control	Manual
Initial State	User is logged into their account and has at least one existing fitness goal.
Inputs	A fitness goal for 1,000 steps is modified to 2,000 steps.
Output	The fitness goal is successfully modified and the updated details are displayed in the user's goal list.
Test Case Derivation	This test covers FUNC-04. The system successfully allows users to edit any of their fitness goals.
How test will be performed	The user will navigate to the fitness goal editing screen. They will modify a fitness goal for 1,000 steps to 2,000 steps and submit the form to update this fitness goal. Then it will be verified that the system correctly displays the updated fitness goal in the user's goal list.

test-FUNC-04-03	**Fitness Goal Creation - Delete Goal**
Control	Manual
Initial State	User is logged into their account and has at least one existing fitness goal.
Inputs	A fitness goal for 1,000 steps is deleted.
Output	The fitness goal is successfully deleted and removed from the user's goal list.
Test Case Derivation	This test covers FUNC-04. The system successfully allows users to delete any of their fitness goals.
How test will be performed	The user will navigate to the fitness goal editing screen. They will delete an existing fitness goal and submit the form to remove this fitness goal. Then it will be verified that the system correctly removed the fitness goal from the user's goal list.

test-FUNC-05	Fitness Goal Tracking
Control	Automatic
Initial State	User is logged into their account and has at least one active fitness goal with 0% progress recorded.
Inputs	Sync activity data that achieves 10% of the goal's progress.
Output	The fitness goal successfully updates to 10% completion upon the next sync.
Test Case Derivation	This test covers FUNC-05. The system successfully syncs fitness data from third-party sources and tracks the progress of any active fitness goals.
How test will be performed	The user will navigate to the fitness goal tracking screen. The system will automatically update the progress of the fitness goal based on the synced activity data. Then it will be verified that the system correctly displays the updated progress in the user's goal list.

4.1.3 Rewards and Virtual Currency

These tests cover awarding virtual currency for activity and keeping each user's balance.

Table 5: Requirements covered: Rewards and Virtual Currency

SRS ID	Requirement
FUNC-06	Reward Allocation
FUNC-07	Virtual Currency Tracking

test-FUNC-06	**Reward Allocation**
Control	Automatic
Initial State	User is logged into their account and has completed at least one existing fitness goal that has yet to be redeemed for virtual currency.
Inputs	An active fitness goal is completed, and the user calls the redeem rewards action.
Output	The system automatically allocates the specified virtual currency to the user's account as a reward for completing the fitness goal. The user's virtual currency balance is increased accordingly and displayed correctly.
Test Case Derivation	This test covers FUNC-06. The system successfully allocates virtual currency as a reward for completing fitness goals.
How test will be performed	After syncing, the user will navigate to the fitness goal tracking screen, select "Redeem Reward", and the system will automatically update the user's virtual currency balance. Then it will be verified that the system correctly displays the updated balance increase in the user's account - ensuring that any virtual balance calculations are accurate and verified.

test-FUNC-07	Virtual Currency Tracking
Control	Manual
Initial State	User is logged into their account and has a positive virtual currency balance.
Inputs	The user navigates to any screen that displays their current virtual currency balance.
Output	On all applicable screens, the system displays the user's current positive virtual currency balance that matches the value stored in the database for the logged in user.
Test Case Derivation	This test covers FUNC-07. The system correctly displays the user's virtual currency balance on any screen where it is applicable, allowing the user to be able to view their accurate balance at any time.
How test will be performed	Navigate to all screens that display the virtual currency balance and verify that all displayed values match the value stored in the database for the current logged in user.

4.1.4 Simulated Investing

These tests cover the portfolio and trading features of the simulated market.

Table 6: Requirements covered: Simulated Investing

SRS ID	Requirement
FUNC-08	Portfolio Creation
FUNC-09	Asset Purchase
FUNC-10	Asset Sale
FUNC-11	Transaction Recording
FUNC-12	Portfolio Performance Calculation

test-FUNC-08	**Portfolio Creation**
Control	Automatic
Initial State	No user account exists.
Inputs	A new user account is successfully registered.
Output	A portfolio is automatically created and associated with the new user account. The `PortfolioID` is generated and stored under this new user account. The newly created portfolio is initialized with no assets.
Test Case Derivation	This test covers FUNC-08. The system will automatically create a new portfolio when a new user account is established.
How test will be performed	A new user will be registered, the system checks if a portfolio has already been created for them. If not, the system will create one. It will then be verified that the portfolio exists in the database and correctly linked to the newly registered user.

test-FUNC-09	**Asset Purchase**
Control	Automatic
Initial State	The user is logged in and has enough virtual currency to purchase the asset.
Inputs	The `UserID`, the `AssetID`, and a call to the buy action.
Output	The cost of the asset is deducted from the user's currency balance, and the asset is added to the user's portfolio.
Test Case Derivation	This test covers FUNC-09. Users can purchase available assets if they have sufficient virtual currency, so the balance must decrease by the asset's cost and the asset must appear in the portfolio.
How test will be performed	After the purchase of an asset, the balance is checked to confirm the currency was removed and the portfolio is checked to confirm the asset was added.

test-FUNC-10	**Asset Sale**
Control	Automatic
Initial State	The user is logged in and holds at least one previously purchased asset in their portfolio.
Inputs	The `UserID`, the `AssetID`, and a call to the sell action.
Output	The sale value is added to the user's currency balance, and the asset is removed from the portfolio.
Test Case Derivation	This test covers FUNC-10. Users can sell assets currently held in their portfolio, so the proceeds must be credited and the holding removed.
How test will be performed	After the sale of an asset, the balance is checked to confirm the currency was added and the portfolio is checked to confirm the asset was removed.

test-FUNC-11	**Transaction Recording**
Control	Manual
Initial State	The user is logged in and has a non-empty transaction history.
Inputs	The `UserID`, and a request to view previous market transactions.
Output	A list of all transactions the user has made, including the type (buy or sell), asset, quantity, price and time.
Test Case Derivation	This test covers FUNC-11, and also confirms the effects of FUNC-09 and FUNC-10. Every purchase and sale generates a transaction record the user can view at any time, so the list must match the trades that were made.
How test will be performed	After completing one or more transactions on an account, the user's transaction list is viewed and checked against the trades that were made.

test-FUNC-12	Portfolio Performance Calculation
Control	Automatic
Initial State	The user is logged in and has a portfolio containing at least one asset.
Inputs	The `UserID`, and a request to retrieve all assets in the portfolio and calculate performance.
Output	The portfolio's current value and its percentage gain or loss. A graph of growth or loss over time is shown to the user.
Test Case Derivation	This test covers FUNC-12. Users can view their current portfolio value and unrealized gains or losses, which helps them judge how well their investment strategies have performed.
How test will be performed	Through manual inspection and the use of financial calculators, the displayed portfolio value, percentage gain or loss, and growth graph are measured and verified.

4.1.5 Community and Leaderboards

These tests cover friends lists and public and private leaderboard groups.

Table 7: Requirements covered: Community and Leaderboards

SRS ID	Requirement
FUNC-13	Leaderboard Participation
FUNC-14	Friend Management
FUNC-15	Private Leaderboards

test-FUNC-13	**Leaderboard Participation**
Control	Manual
Initial State	The user is logged in, and a leaderboard with open spaces is available to join.
Inputs	The `UserID` and the `LeaderboardID` of a leaderboard with open spaces.
Output	The user joins the intended leaderboard and appears in its member list.
Test Case Derivation	This test covers FUNC-13. A user can join a group, so after the join request the user must be a member of the chosen leaderboard and able to compete with its other members.
How test will be performed	After the user attempts to join the leaderboard, check that the user's `UserID` is linked to that `LeaderboardID`.

test-FUNC-14	**Friend Management**
Control	Automatic
Initial State	The user is logged in and can search for other users by `UserID`.
Inputs	The `UserID` of another application user.
Output	A friend request is sent to that user, and the invitee can respond to it.
Test Case Derivation	This test covers FUNC-14. Users can add and remove friends, so a request must reach the invitee, and the result of their response (approved or denied) must be reflected in both users' friends lists. This gives users the social aspect of comparing progress with selected friends.
How test will be performed	After the user sends a friend request, check that both the requester and the invitee can see the request and, once the invitee responds, the result of the action (approved or denied).

test-FUNC-15	Private Leaderboard Creation
Control	Automatic
Initial State	The user is logged in.
Inputs	A valid `UserID` and a request to create a new leaderboard.
Output	A new private leaderboard is created with a unique `LeaderboardID`.
Test Case Derivation	This test covers FUNC-15. Users can create private competition groups, so a successful request must produce a new group with its own ID. The created group is also needed by the FR-13 tests, which require a leaderboard to join.
How test will be performed	After the user completes the steps to create a leaderboard, check that a valid `LeaderboardID` exists and that the creator is a member of the new leaderboard.

4.1.6 Progress Reporting

These tests cover the summaries of activity, goals, earnings, portfolio performance, and standings.

Table 8: Requirements covered: Progress Reporting

SRS ID	Requirement
FUNC-16	Progress Reporting

test-FUNC-16	Progress Reporting
Control	Automatic
Initial State	The user is logged in and has a trackable history of activity, goals, currency earnings and portfolio activity.
Inputs	A valid `UserID` and a request to view the progress data connected to that ID.
Output	Current and historical summaries of the user's fitness activity, goal completion, virtual currency earnings, portfolio performance and leaderboard standings, each matching the data recorded for that user.
Test Case Derivation	This test covers FUNC-16. Users can access current and historical progress information at any time, so each summary must match the stored data. It also verifies the combined effects of FUNC-03, FUNC-04, FUNC-05, FUNC-06, FUNC-07 and FUNC-12, whose data feed the report.
How test will be performed	After the `UserID` is submitted to retrieve account progress, check that the returned summaries include the user's activity, goal tracking, currency earnings, portfolio performance and leaderboard standings if applicable

4.2 Tests for Nonfunctional Requirements

The subsections below follow the nonfunctional requirement sections of the SRS (**?**). Each one lists the SRS subsections and requirement IDs that its tests cover.

Each test case gives its type, initial state, input or condition, expected output or result, and how the test will be performed. Static tests, such as reviews, inspections, and walkthroughs, instead say who is involved and how the review is run.

4.2.1 Look and Feel

These tests cover the Look and Feel Requirements section of the SRS (**?**), spanning appearance and style.

Appearance Covers LF-APP-01 to LF-APP-05. The appearance tests verify that Flexin' delivers an uncluttered visual hierarchy balancing physical activity with simulated financial portfolios, distinguishable compounding data visualizations, responsive layout scaling across diverse Android displays, WCAG-compliant color contrast across Light and Dark themes, and persistent core navigation.

Table 9: Requirements covered: Appearance

SRS ID	**Requirement**	**Test Case ID**
LF-APP-01	Visual Hierarchy	test-LF-APP-01
LF-APP-02	Compounding Chart	test-LF-APP-02
LF-APP-03	Responsive Layout	test-LF-APP-03
LF-APP-04	Theme and Contrast	test-LF-APP-04
LF-APP-05	Navigation	test-LF-APP-05

test-LF-APP-01	Visual Hierarchy (Dashboard Balance)
Type	Dynamic, Manual (Timed Usability Test)
Initial State	The application is running on a physical Android test device or emulator. The test user account is fully logged in and populated with active fitness metrics (e.g., current weekly workout and step goal progress) and simulated investment portfolio data (e.g., total portfolio valuation and daily gain/loss). The application is opened to the main Dashboard screen.
Input/Condition	Present a participant from the target user cohort with the populated Dashboard and deliver the standardized verbal prompt: "Locate your current fitness progress and your overall portfolio value."
Output/Result	At least UsabilityPassRate of participants in the testing cohort locate and point out both their current fitness progress and their portfolio value within LocateTime without prompting or external assistance.
How test will be performed	Conduct individual usability evaluation sessions with a cohort of at least MinCohortSize participants matching the target demographic. A test administrator starts a digital stopwatch and video/screen capture upon completing the verbal task prompt and stops timing once the participant verbally points to or touches both target summary cards. The elapsed time and success status (pass if within LocateTime, fail otherwise) are recorded in the usability test log.

test-LF-APP-02	Compounding Chart (Trace Distinction)
Type	Dynamic, Manual (Usability Visual Identification Test)
Initial State	The application is running and navigated to the Portfolio Compounding Chart view. The compounding chart displays dual historical/projected traces: one representing cumulative workout earnings (contributions) and the other representing simulated investment returns (capital gains and dividends). The chart includes a visible legend.
Input/Condition	Open the Portfolio compounding chart view and instruct the participant: "Identify and distinguish which visual trace represents your workout earnings and which represents your investment gains/dividends."
Output/Result	Participant correctly identifies and distinguishes between the two visual traces within ChartReadTime. The visual traces must exhibit distinct color hues (tested for color-blind accessibility) and distinct visual line patterns (e.g., solid vs. dashed line), supported by an unambiguous legend.
How test will be performed	Administered during participant usability sessions. The evaluator records participant accuracy, time-to-identification via stopwatch, and qualitative remarks regarding chart readability and trace distinction. Both quantitative identification speed and qualitative feedback are documented in the test report.

test-LF-APP-03	**Responsive Layout (Screen Sizes and Font Scaling)**
Type	Dynamic, Semi-Automated / Manual (Multi-Device Emulator and Device Matrix)
Initial State	Application build deployed across an emulator/device matrix encompassing Android 10+ (API level 29+) with physical screen sizes ranging from 5.5" to 6.8" (including Google Pixel 4a, Pixel 7, and Samsung Galaxy S21 Ultra profiles). System font scaling in Android Accessibility settings is configured to TestFontScale.
Input/Condition	Launch the application on each device configuration and systematically navigate through all primary views (Dashboard, Activity, Market, Learn, Community) and key secondary modal/form dialogs.
Output/Result	Zero text truncations, text clipping, or unreadable overlapping text across all screens. Zero overlapping interactive UI elements. All touch targets maintain a minimum interactive hit area of 48×48 dp in accordance with UHR-ACC-04.
How test will be performed	Automated Detox tests on each emulator profile read the on-screen bounds of every interactive element to assert that touch targets meet the $\geq 48 \times 48$ dp constraint and verify layout bounds do not collide. Additionally, a manual visual inspection across the multi-device emulator matrix is conducted by two QA team members using a responsive layout verification checklist, logging any visual clipping defects into GitHub Issues.

test-LF-APP-04	**Theme and Contrast (Light/Dark Mode and Indicators)**
Type	Static / Automated Tool-Assisted Accessibility Audit
Initial State	Application installed with both Light and Dark theme palettes fully implemented. Mock data populated displaying positive/negative financial returns and active/completed fitness milestones.
Input/Condition	Toggle the device system theme between Light mode and Dark mode across all screens. Render all fitness status indicators (e.g., progress rings, streak badges) and financial status indicators (e.g., asset performance, gain/loss markers).
Output/Result	All text and interactive graphical components meet WCAG 2.1 AA contrast requirements: contrast ratio $\geq 4.5:1$ for normal body text and $\geq 3:1$ for large text ($\geq$ 18 pt or bold $\geq$ 14 pt) and graphical icons, complying with COMP-STD-02. Fitness and financial status indicators remain readily distinguishable in both Light and Dark modes without semantic confusion.
How test will be performed	The accessibility audit is conducted by two QA team members using automated scanning tools (Google Accessibility Scanner for Android and the Colour Contrast Analyser tool) on captured UI screen frames. Automated tool reports are cross-checked against WCAG 2.1 Success Criteria 1.4.3 (Contrast Minimum) and 1.4.11 (Non-text Contrast). Any color contrast violations or indistinguishable state indicators are logged as critical UI bugs.

test-LF-APP-05	Navigation (Persistent Bottom Bar)
Type	Dynamic, Automated UI Test
Initial State	Application launched in a clean state with an authenticated user session.
Input/Condition	Programmatically trigger navigation through each of the five top-level primary screens: Dashboard, Activity, Market/Portfolio, Learn, and Community.
Output/Result	The persistent bottom navigation bar is present and visible on every primary screen. The bar contains exactly five navigation destinations with correct iconography and labels (Dashboard, Activity, Market, Learn, Community). The active destination is distinctly highlighted corresponding to the active route.
How test will be performed	Automated Detox end-to-end test written in TypeScript. The test script taps each navigation destination in sequence, asserting that the element with test ID `bottom_navigation_bar` remains visible (`toBeVisible()`) and that the tab for the active screen is marked selected in its accessibility state while the other four are not.

Style Covers LF-STY-01 to LF-STY-04. The style tests verify that Flexin' maintains an approachable and encouraging tone, ensures strict virtual currency branding to prevent confusion with real-world financial assets, provides non-intrusive gamification reward animations, and enforces component consistency across the shared design system.

Table 10: Requirements covered: Style

SRS ID	Requirement	Test Case ID
LF-STY-01	Approachable Tone	test-LF-STY-01
LF-STY-02	Virtual Currency Symbol	test-LF-STY-02
LF-STY-03	Non-Intrusive Gamification	test-LF-STY-03
LF-STY-04	Component Consistency	test-LF-STY-04

test-LF-STY-01	Approachable Tone (Typography, Terminology, and Tooltips)
Type	Static / User Survey Evaluation
Initial State	A representative testing cohort ($N \geq 10$) matching the target user demographic (novice investors and general fitness users) is assembled. A staging build of Flexin' is loaded on test devices with simulated market and fitness data.
Input/Condition	Participants complete a self-directed walkthrough of the financial education, trading, and portfolio tracking screens, followed by completing the structured post-test usability survey defined in Appendix 6.2.
Output/Result	At least UsabilityPassRate of target-demographic evaluators rate the application's interface tone as approachable, supportive, and non-intimidating (Likert score $\geq$ 4 out of 5 on survey questions Q1–Q3). Contextual explanatory tooltips are confirmed present and functional for 100% of complex financial terms (e.g., "dividends", "unrealized return", "expense ratio").
How test will be performed	Usability evaluators administer the survey questionnaire immediately after the participant walkthrough. Survey ratings are compiled to calculate the proportion of positive ratings ($\geq 4/5$). In addition, a static inspection of all financial UI screens is performed by two team members against a pre-approved glossary of financial terminology to verify that every designated financial term displays an interactive tooltip icon that renders plain-language definitions upon tap.

test-LF-STY-02	**Virtual Currency Symbol (FLX vs. Real-World Currency)**
Type	Static Code/String Audit and Dynamic UI Inspection
Initial State	Clean repository workspace on the release candidate branch. Application installed on a test device with simulated user balances and market quotes displayed.
Input/Condition	Execute automated static regex and linter scripts across all application source code, string resources (`strings.xml`), and currency formatters. Perform manual inspection across all views displaying balances, prices, rewards, and portfolio values.
Output/Result	Zero occurrences of real-world fiat currency symbols ($, £, €, ¥) associated with virtual currency amounts. 100% compliance with the application's virtual currency symbol (FLX). Prominent simulated market disclaimers remain visible in accordance with PR-SC-03.
How test will be performed	1. *Static Analysis*: A continuous integration (CI) script executes a static regex search over the UI codebase and XML localization resources to verify that no hardcoded fiat currency symbols ($, £, €, ¥) exist in balance formatting paths. 2. *Dynamic Inspection*: A QA engineer performs an end-to-end screen walkthrough verifying that every balance indicator on the Dashboard, Market, Activity, and History screens explicitly displays the "FLX" label.

test-LF-STY-03	**Non-Intrusive Gamification (Reward Animations)**
Type	Dynamic, Automated / Manual Timing Test
Initial State	Application installed and running. Test account configured with a near-complete milestone (e.g., logging a completed workout that triggers an achievement celebration).
Input/Condition	Trigger a milestone reward celebration animation. Evaluate two test conditions: • Condition A: Allow the animation to play to completion without user touch input. • Condition B: Tap the background area or perform a swipe gesture while the animation is actively rendering.
Output/Result	• Condition A: Total animation playback duration does not exceed MaxAnimationTime, smoothly returning control to the standard view. • Condition B: Tapping the background or swiping immediately dismisses the animation within DismissTime, immediately restoring user interaction to the underlying interface.
How test will be performed	Automated Jest tests with React Native Testing Library use Jest's fake timers to advance the animation clock and assert the animation start, dismiss, and end times. In addition, manual verification using a high-frame-rate (60 fps) screen recording is conducted to measure the frame delay between user tap interrupt and visual dismissal.

test-LF-STY-04	Component Consistency (Style Guide Compliance)
Type	Static Inspection (Design Walkthrough)
Initial State	Complete design system specification (Figma component library and the app's shared theme tokens in `src/mobile/src/theme`) approved by the UI/UX lead. Staging build of the application containing all user-facing screens.
Input/Condition	Compare all UI components (buttons, forms/input fields, modal dialogs, contextual tooltips, summary cards, and toggle switches) across every screen against the shared design system specifications.
Output/Result	Zero discrepancies, undocumented custom components, or unstandardized styling found. All UI elements adhere to defined typography, border radiuses, color tokens, and elevation levels.
How test will be performed	A formal peer-review inspection meeting is conducted by three team members: the UI/UX designer, the lead frontend developer, and a QA reviewer. The review team steps through each screen using a structured Component Consistency Checklist covering buttons, form controls, modals, tooltips, cards, and toggles. Any identified departures from the style guide are documented with screenshot evidence and logged as defects in GitHub Issues.

4.2.2 Usability and Humanity

These tests cover the Usability and Humanity Requirements section of the SRS.

Ease of Use Covers UHR-EOU-01 to UHR-EOU-09.

test-UHR-EOU-01	First-Time User Workflow Evaluation
Type	Dynamic, Manual, User Testing
Initial State	New participant has no prior experience with Flexin'.
Input/Condition	Participant creates an account and attempts to access key application functionality (complete onboarding, access their account dashboard and details, and navigate between fitness and financial focused screens).
Output/Result	Participant completes onboarding within OnboardingTime, can access dashboard information immediately after login, and can reach detailed portfolio and fitness management features within two taps from the dashboard.
Covers	UHR-EOU-01, UHR-EOU-02, UHR-EOU-03, and UHR-EOU-08.
How test will be performed	Participants will be observed while creating an account, completing onboarding, locating dashboard information, and navigating to portfolio management, activity tracking, and currency transaction screens. Completion times and navigation paths will be recorded by the observer.

test-UHR-EOU-04	**User Feedback and Error Recovery Evaluation**
Type	Dynamic, Manual, User Testing
Initial State	User is logged into the application.
Input/Condition	User performs both successful and unsuccessful actions such as invalid login attempts, failed transactions, goal updates, portfolio transactions, and profile modifications.
Output/Result	Appropriate error messages appear immediately with corrective guidance with invalid or unsuccessful actions. Successful actions display confirmation feedback within two seconds.
Covers	UHR-EOU-04 and UHR-EOU-05.
How test will be performed	Testers will intentionally trigger errors and complete successful actions throughout the application. All displayed messaging and feedback timing will be verified against the requirement specifications.

test-UHR-EOU-07	**Consistency Review**
Type	Static Code and UI Review, Manual
Initial State	All UI screens are developed.
Input/Condition	User navigates through the application screens to check for consistency in design and functionality.
Output/Result	The interface remains visually and behaviorally consistent across screens.
Covers	UHR-EOU-07.
How test will be performed	Testers will perform a design consistency review examining terminology, navigation, icons, colors, and interaction patterns across the application - listing any inconsistencies found.

test-UHR-EOU-09	Re-Learnability Study
Type	Dynamic, Manual, User Testing
Initial State	A user exists that has previously used the application consistently, but has been an inactive user for at least InactivityPeriod.
Input/Condition	Participants attempt to perform representative fitness and investing actions.
Output/Result	Returning users regain proficiency within ReLearnTime.
Covers	UHR-EOU-09.
How test will be performed	Returning users will be observed as they attempt a predefined set of core application tasks (i.e. manage a fitness goal, manage their financial portfolio by buying/selling an asset). Task completion times and success rates will be measured and compared by the observer.

Personalization and Internationalization Covers UHR-PER-01 to UHR-PER-04, UHR-INT-01.

test-UHR-PER-01	Personalization and Internationalization Verification
Type	Dynamic, Manual
Initial State	User account exists and user is logged in.
Input/Condition	User customizes goals, notifications, privacy settings, competition groups, or uses the application in different Canadian time zones.
Output/Result	All personalization options take effect immediately and correctly, and daily calculations reflect the user's local Canadian time zone.
Covers	UHR-PER-01, UHR-PER-02, UHR-PER-03, UHR-PER-04, and UHR-INT-01.
How test will be performed	Testers will modify each available customization setting and verify corresponding system behavior. Device time zones will be changed to confirm proper local calculation of goals, rewards, and streaks.

Learning Covers UHR-LRN-01 to UHR-LRN-04.

test-UHR-LRN-01	**Learning and Educational Effectiveness Evaluation**
Type	Dynamic, Manual, User Testing
Initial State	First-time users have no formal training.
Input/Condition	Participants use Flexin' to synchronize fitness data, earn currency, make investments, complete fitness goals, and interact with educational material.
Output/Result	At least UsabilityPassRate of participants successfully complete core fitness and financial tasks without external assistance and rate financial and fitness term explanations as clear (4 or 5 on a 5-point scale).
Covers	UHR-LRN-01, UHR-LRN-02, UHR-LRN-03, and UHR-LRN-04.
How test will be performed	Participants will complete structured fitness and investing activities. Observers will record completion success rates, assistance requirements, and participant survey responses regarding educational content clarity.

Understandability and Politeness Covers UHR-UND-01 to UHR-UND-03, UHR-POL-01 to UHR-POL-05.

test-UHR-UND-01	Understandability Review
Type	Static Review and Dynamic Testing
Initial State	User-facing content has been implemented.
Input/Condition	Review all terminology, educational content, portfolio views, and transaction feedback screens.
Output/Result	Information is presented in plain language, portfolio contributions from completing fitness goals and investment gains are clearly distinguished, and outcomes affecting fitness progress, virtual currency balances, and portfolio holdings are immediately communicated to the user.
Covers	UHR-UND-01, UHR-UND-02, and UHR-UND-03.
How test will be performed	Reviewers will inspect all user-facing content thoroughly.

test-UHR-POL-01	Respectful Communication and Content Moderation Review
Type	Static Review and Dynamic Testing
Initial State	User-facing messaging and community features are implemented.
Input/Condition	Review notifications, leaderboards, feedback messages, performance-related messaging, and public profile creation workflows.
Output/Result	Communication remains respectful, supportive, non-punitive, and offensive user-generated content is rejected.
Covers	UHR-POL-01, UHR-POL-02, UHR-POL-03, UHR-POL-04, and UHR-POL-05.
How test will be performed	Reviewers will thoroughly inspect all generated messaging, achievement notifications, leaderboard displays, and failed-goal messaging. Testers will also attempt to create usernames and content containing prohibited language to verify enforcement of moderation rules.

Accessibility Covers UHR-ACC-01 to UHR-ACC-06.

test-UHR-ACC-01	Accessibility Compliance Evaluation
Type	Dynamic, Automated Accessibility Testing
Initial State	Complete application build is available.
Input/Condition	Execute core workflows using accessibility tools and accessibility-focused testing procedures.
Output/Result	Screen readers function correctly, text scaling is supported, information is not color-dependent, controls satisfy touch target requirements, notifications use multiple modalities, and multiple activity types are supported.
Covers	UHR-ACC-01, UHR-ACC-02, UHR-ACC-03, UHR-ACC-04, UHR-ACC-05, and UHR-ACC-06.
How test will be performed	Automated accessibility tools, manual accessibility inspections, screen reader testing, touch target validation, notification testing, and activity tracking validation will be conducted across major workflows.

4.2.3 Performance

These tests cover the Performance Requirements section of the SRS. Timing tests run on the reference phone, which is the Android phone the team uses for every timing measurement. Its model and Android version are recorded in the VnV Report so the results can be repeated. Names in sans serif are symbolic constants, defined in Table 13.

Speed and Latency Covers PR-SL-01 to PR-SL-04.

test-PR-SL-01	App launch time
Type	Dynamic, Manual
Initial State	The release build is installed on the reference phone, the user is signed in, and the app is fully closed (removed from recent apps).
Input/Condition	The tester opens the app from the home screen icon LaunchTrials times, closing it fully between each launch.
Output/Result	The home screen appears within LaunchTime in at least LatencyPassRate of the launches (19 of 20).
How test will be performed	The tester records each launch as a screen recording and measures the time from the icon tap to the first frame where the home screen is fully drawn. All LaunchTrials times are recorded in the VnV Report.

test-PR-SL-02	Response to input
Type	Dynamic, Automatic
Initial State	The release build is installed on the reference phone with a signed-in test account that has activity, goals, and a portfolio.
Input/Condition	A Detox script visits every main screen and presses the main buttons on each one, for at least 100 measured actions in total.
Output/Result	At least LatencyPassRate of the measured transitions and button responses finish within InputResponseTime.
How test will be performed	The script logs the time between each action and the moment the expected element on the next screen becomes visible. A summary script reports the share of actions under InputResponseTime.

test-PR-SL-03	Trade confirmation time
Type	Dynamic, Automatic
Initial State	A signed-in test account has enough currency to buy, and some holdings to sell, in several assets.
Input/Condition	A Detox script places 50 buy orders and 50 sell orders of different sizes and confirms each one.
Output/Result	In at least LatencyPassRate of the trades, the updated holdings and currency balance appear within TradeConfirmTime of confirmation.
How test will be performed	The script records the time from the confirm tap until the new balance and holding quantity are shown, and checks that the shown values match the expected values after the trade.

test-PR-SL-04	Activity sync time
Type	Dynamic, Manual
Initial State	A test account is signed in on the reference phone. Health Connect holds new step and workout records that the app has not synced yet. The app is closed.
Input/Condition	The tester opens the app.
Output/Result	Within SyncTime of opening, the currency balance includes the currency for all of the new Health Connect activity.
How test will be performed	The tester calculates the expected balance from the conversion rules beforehand, opens the app with a timer running, and records when the balance reaches the expected value. This is repeated SyncTrials times with different amounts of new activity.

Safety-Critical Covers PR-SC-01 to PR-SC-08.

test-PR-SC-01	Daily earning limits
Type	Dynamic, Automatic
Initial State	A test account with no activity recorded for the test day.
Input/Condition	Activity records for one day at 0.5, 1, 1.5, 2, and 5 times HealthyDailyActivity, each sent to a fresh test account.
Output/Result	No account earns more than DailyEarnCap for the day. The currency earned per unit of activity above HealthyDailyActivity is lower than the rate below it.
How test will be performed	A JUnit test sends each set of records through the reward service and compares the earned amount against the cap and against the earned amount for the lower levels.

test-PR-SC-02	No penalties for missed days
Type	Dynamic, Automatic and Static
Initial State	A test account with a currency balance, holdings, and an active streak.
Input/Condition	(1) The account records no activity for several days. (2) The user freezes their streak before a missed day.
Output/Result	(1) The balance and holdings are unchanged after the inactive days. (2) The frozen streak is not reset by the missed day.
How test will be performed	A JUnit test advances the server clock over the inactive days and checks the balance, holdings, and streak. A code review of every place that writes to balances and holdings confirms that none of them lowers a value because of inactivity.

test-PR-SC-03	Simulated market notice
Type	Static, Manual inspection
Initial State	The release build with a test account that holds assets.
Input/Condition	The tester visits every screen that shows prices, holdings, or returns.
Output/Result	Each of these screens shows the notice that the market is simulated and that results do not predict real returns, and links to the explanation of what the simulation does not model.
How test will be performed	Two team members go through a checklist of all market and portfolio screens taken from the app's navigation. For each screen they record whether the notice is visible and whether the link opens the explanation.

test-PR-SC-04	Not financial advice statement
Type	Dynamic, Automatic and Static
Initial State	A fresh install of the app with no account.
Input/Condition	The tester tries to finish sign-up without acknowledging the statement, then acknowledges it and finishes sign-up, then opens settings.
Output/Result	Sign-up cannot continue until the statement is acknowledged, and the statement can be opened from settings.
How test will be performed	A Detox script checks the sign-up and settings behaviour. Separately, a team member reviews all lessons, tips, and portfolio insights against the list in the SRS and records any text that recommends a specific real-world investment. The test passes when none is found.

test-PR-SC-05	Exercise injury warning
Type	Dynamic, Automatic
Initial State	A fresh install of the app with no account.
Input/Condition	The tester tries to finish sign-up without acknowledging the injury warning and the terms of use, then acknowledges both, finishes sign-up, and opens settings.
Output/Result	Sign-up cannot continue until both are acknowledged, and both can be opened from settings.
How test will be performed	A Detox script runs the sign-up flow with each acknowledgement left out in turn, then checks that settings links to both.

test-PR-SC-06	Safe goal progression
Type	Dynamic, Automatic
Initial State	A test account with an automatically raised goal plan.
Input/Condition	(1) The goal service generates eight weeks of goals after the user meets every goal. (2) The user misses MissedGoalLimit goals in a row. (3) The user lowers a goal from the goal screen.
Output/Result	(1) No weekly goal is more than MaxGoalIncrease above the week before, and every week has at least one rest day. (2) The app suggests a lower goal. (3) The lower goal is saved and used.
How test will be performed	JUnit tests check the generated plan and the suggestion. A Detox script lowers a goal from the goal screen and checks the saved value.

test-PR-SC-07	Notification limits
Type	Dynamic, Automatic
Initial State	A test account with notifications on and quiet hours set.
Input/Condition	Events that would send goal-related notifications are created throughout a simulated day, including during quiet hours.
Output/Result	No notifications are sent during quiet hours, and no more than MaxDailyNotifications goal-related notifications are sent in the day. After the user raises the limit, the new limit is used.
How test will be performed	A JUnit test runs the notification scheduler with a fixed clock and counts the notifications it sends and the times it sends them.

test-PR-SC-08	Safe leaderboard rules
Type	Dynamic, Automatic
Initial State	A leaderboard group with several test users who have different goals and different amounts of activity.
Input/Condition	The leaderboard is computed, one user leaves the group, and the leaderboard is computed again. A new account is also created.
Output/Result	Rankings are ordered by each user's percentage of their goal, not by raw activity. The user who left no longer appears. No user's balance changes because of their ranking. The new account is not on any leaderboard until the user opts in.
How test will be performed	A JUnit test sets up users where ranking by raw activity and ranking by goal percentage give different orders, then checks the order, the membership, and every balance before and after.

Precision or Accuracy Covers PR-PA-01 to PR-PA-06.

test-PR-PA-01	**Exact money values**
Type	Dynamic, Automatic
Initial State	A test account with a known starting balance.
Input/Condition	A scripted sequence of 1,000 buy and sell trades at prices that do not divide evenly, such as 33.33 and 0.07.
Output/Result	The stored balance after the sequence matches the expected balance exactly, and every amount shown in the app has two decimal places.
How test will be performed	A JUnit test runs the trades through the trading service and compares the final balance with a value computed separately using exact decimal arithmetic. A Detox check reads the displayed balance and trade amounts and checks their format.

test-PR-PA-02	**Consistent currency awards**
Type	Dynamic, Automatic
Initial State	An empty test account for each activity record set.
Input/Condition	A fixed set of test activity records covering steps, workouts of each type, and edge values such as zero activity. Each set is processed twice.
Output/Result	The awarded currency matches the hand-calculated amount from the published conversion rules, and both runs award the same amount.
How test will be performed	The team calculates the expected amounts by hand and stores them as test fixtures. A JUnit test compares the reward service's output against them.

test-PR-PA-03	No double counting
Type	Dynamic, Automatic
Initial State	A test account with two connected sources.
Input/Condition	The same walk is reported by both sources with overlapping times. A second case has two separate walks, one from each source, that do not overlap.
Output/Result	The overlapping walk earns the amount for one recording. The two separate walks each earn currency.
How test will be performed	A JUnit test sends both cases through the import and reward services and compares the awards with the expected amounts.

test-PR-PA-04	Portfolio value
Type	Dynamic, Automatic
Initial State	Several test portfolios with different holdings, fixed simulated prices, and some uninvested currency.
Input/Condition	The portfolio screen is opened for each test portfolio.
Output/Result	The displayed value equals the sum of quantity times price for each holding, plus uninvested currency, to the cent.
How test will be performed	The expected values are calculated by hand. A JUnit test checks the value returned by the API, and a Detox script checks the value shown on screen.

test-PR-PA-05	Workout dates and late syncs
Type	Dynamic, Automatic
Initial State	A test account with an active streak.
Input/Condition	(1) A workout done at 11:30 pm is synced after midnight. (2) A workout recorded in a different time zone is synced. (3) A workout from two days ago arrives after the streak was already marked as broken.
Output/Result	Each workout is recorded on the day it happened on the source device. In case (3) the streak and the rewards for that day are recalculated to include the late workout.
How test will be performed	A JUnit test sends records with set source timestamps and time zones, using a fixed server clock, then checks the stored dates, the streak, and the rewards.

test-PR-PA-06	Realistic market simulation
Type	Dynamic, Automatic
Initial State	The market simulation with its calibrated settings and a fixed random seed.
Input/Condition	The simulation is run over at least 30 simulated years for a broad market asset, with several different seeds.
Output/Result	The average annual return and the annual volatility are consistent with historical data for a broad market index, and each run includes downturns.
How test will be performed	A script computes the average annual return, annual volatility, and largest drop for each run and compares them with the same measures from the historical data used for calibration. The acceptable ranges are agreed with the supervisor before the test and recorded in the VnV Report.

Robustness or Fault-Tolerance Covers PR-RFT-01 to PR-RFT-04.

test-PR-RFT-01	Offline activity
Type	Dynamic, Manual
Initial State	A test account signed in on the reference phone, synced, then put in airplane mode with the app closed.
Input/Condition	Activity is recorded in Health Connect over a gap of OfflineSyncWindow (set by moving the activity timestamps back, or by waiting out the period). The phone is then reconnected and the app is opened.
Output/Result	All of the activity from the gap is credited at the next sync.
How test will be performed	The tester compares the credited currency with the amount calculated by hand for the recorded activity.

test-PR-RFT-02	Atomic writes
Type	Dynamic, Automatic
Initial State	A test account with a known balance and holdings.
Input/Condition	(1) A failure is injected at each step of every multi-step write: trades, reward credits, and progress updates. (2) The connection or server is cut in the middle of a test trade. (3) 100 trade requests are sent at the same time against one account.
Output/Result	(1) Each failed write leaves the data as it was. (2) After the cut, the balance and holdings match either the state before the trade or the state after it, and a completed trade used the confirmed price. (3) Every request is either fully completed or fully rolled back, and the balance is never negative.
How test will be performed	JUnit tests use fault injection in the service layer to fail each step in turn. The concurrent test sends the 100 requests from parallel threads, then checks that the final balance and holdings equal the starting state plus the completed trades.

test-PR-RFT-03	Failing data sources
Type	Dynamic, Automatic
Initial State	A test account with two connected sources.
Input/Condition	One source is made unavailable. In a second run, it returns malformed data.
Output/Result	The app stays usable with the other source, retries the failed sync, then shows a message naming the failing source and the cause. No currency is awarded for the malformed data.
How test will be performed	The failing source is replaced with a mock that times out or returns malformed records. Backend tests check the retries and the awards, and a Detox script checks the app and the message.

test-PR-RFT-04	Backend unavailable
Type	Dynamic, Manual
Initial State	A test account that has synced at least once on the reference phone.
Input/Condition	The backend is stopped, then the app is opened.
Output/Result	The app opens without crashing, shows the last known balance and portfolio, and shows a notice that the data may be out of date.
How test will be performed	The tester stops the backend, opens the app, visits each main screen, and records whether any screen crashes or is empty.

Capacity Covers PR-C-01, PR-C-02.

test-PR-C-01	Concurrent users
Type	Dynamic, Automatic
Initial State	The backend is deployed to the test server with 500 registered test accounts.
Input/Condition	A load test runs 100 simulated users at the same time. Each one browses screens and places trades.
Output/Result	Response times during the load test meet PR-SL-02 and PR-SL-03.
How test will be performed	The load testing tool sends the API calls that the app makes for each action and records the response times. While the load runs, test-PR-SL-02 and test-PR-SL-03 are also run on the reference phone.

test-PR-C-02	Two years of history
Type	Dynamic, Automatic
Initial State	A test account loaded with two years of daily activity and weekly trades.
Input/Condition	The user opens the activity history, trade history, and progress screens and scrolls to the oldest records.
Output/Result	The full two years of history is shown, and the screens meet PR-SL-02.
How test will be performed	A seeding script creates the test data. A Detox script opens each history screen, checks that the oldest records are shown, and times each transition.

Scalability or Extensibility Covers PR-SE-01 to PR-SE-03.

test-PR-SE-01	Adding a fitness source
Type	Static, Code inspection
Initial State	The code with the fitness sources available at release.
Input/Condition	The team adds a mock fitness source.
Output/Result	All of the changes needed are inside the integration module, and the currency conversion code is unchanged.
How test will be performed	A team member adds the mock source on a branch. The reviewers check the diff to confirm that every changed file is inside the integration module.

test-PR-SE-02	Adding an asset type
Type	Static, Code inspection
Initial State	The code with stocks, bonds, and ETFs.
Input/Condition	The team adds a new test asset type.
Output/Result	The change only adds asset definitions and market engine configuration. The buy and sell code is unchanged, and the existing trading tests pass with the new asset type.
How test will be performed	A team member adds the asset type on a branch. The reviewers check the diff, and the trading tests are run with the new asset type.

test-PR-SE-03	Adding server capacity
Type	Dynamic, Automatic
Initial State	The backend is deployed with extra server capacity, and the app version is the same one used in test-PR-C-01.
Input/Condition	The load test from test-PR-C-01 is run with 200 simulated users.
Output/Result	Response times meet PR-SL-02 and PR-SL-03, with no change to the mobile app.
How test will be performed	The team adds server capacity, reruns the load test at double the load, and confirms that the app's version number is the same as in test-PR-C-01.

Longevity Covers PR-L-01, PR-L-02.

test-PR-L-01	**Supported Android versions**
Type	Static and Dynamic, Automatic
Initial State	The release build.
Input/Condition	(1) The target Android version in the build settings is compared with Google Play's requirement for new apps at release time. (2) The test suite is run on an emulator with the newest Android version available in April 2027.
Output/Result	The build meets Google Play's target version requirement, and the test suite passes on the newest Android version.
How test will be performed	A team member checks the build settings against Google Play's published requirement before release. The Detox suite is run on the newest Android emulator image before the final demo.

test-PR-L-02	**Keeping user history**
Type	Static, Code inspection
Initial State	The backend code, database schema, and server configuration.
Input/Condition	The team searches for every scheduled job, database event, and retention setting that deletes data.
Output/Result	None of them deletes activity or portfolio history for an active account. History is deleted only when the user deletes it or deletes their account.
How test will be performed	A team member lists every delete path in the code and database configuration. A second team member reviews the list and confirms that each one is started by the user.

4.2.4 Operational and Environmental

These tests cover the Operational and Environmental Requirements section of the SRS.

Productization Covers PROD-01, PROD-02.

Table 11: Requirements covered: Productization

SRS ID	Requirement
PROD-01	Build and Quality Assurance
PROD-02	Installability

test-PROD-01	Build and Quality Assurance
Type	Dynamic, Automatic
Initial State	The final version of the app is in a pre-final revision and has been tested.
Input/Condition	The internally developed test suite and the Android pre-release test package.
Output/Result	All tests pass with no warnings caused by technical issues.
Covers	PROD-01. By passing these tests, the development team receives confirmation that the application is ready for release.
How test will be performed	The test is completed through the automated test suites and the process of publishing the app to the Android app store.

test-PROD-02	Public Store Installation
Type	Dynamic, Manual
Initial State	The app is published on a publicly accessible Google Play Store listing.
Input/Condition	The user attempts to find and install the application.
Output/Result	The app is installed and ready for the user to use.
Covers	PROD-01, and verifies the effects of PROD-02. Users must be able to access, download, install and update the app from a public source.
How test will be performed	Users are asked to install the app on a phone before launch. This shows how familiar users are with finding applications in the app store and confirms that the app is accessible.

4.2.5 Maintainability and Support

These tests cover the Maintainability and Support Requirements section of the SRS.

Maintenance Covers MAS-MNT-01 to MAS-MNT-05.

test-MAS-MNT-01	Fitness Activity Addition Effort
Type	Static Code Review, Manual
Initial State	System supports existing reward-eligible fitness activities.
Input/Condition	Development team adds a new fitness activity type (e.g., swimming) to the application.
Output/Result	The new activity type is implemented, deployed, and fully tested within FeatureAddTime.
Covers	MAS-MNT-01 and verifies that new reward-eligible fitness activities can be added with minimal maintenance effort.
How test will be performed	The development team will implement a new fitness activity type using the existing system architecture. Project records, pull requests, testing evidence, and deployment dates will be reviewed to verify completion within FeatureAddTime.

test-MAS-MNT-02	Investment Asset Addition Effort
Type	Static Code Review, Manual
Initial State	System contains an existing set of fictional investment assets.
Input/Condition	Development team adds a new investment asset or asset type to the simulated market.
Output/Result	The new investment asset is implemented, deployed, and fully tested within FeatureAddTime.
Covers	MAS-MNT-02 and verifies that additional asset types can be introduced without major redevelopment effort.
How test will be performed	A new fictional investment asset will be implemented and integrated into the market engine. Development records and deployment history will be reviewed to verify completion within FeatureAddTime.

test-MAS-MNT-03	Corrective Maintenance Effort
Type	Static Review, Manual
Initial State	Critical production defect has been identified by a user.
Input/Condition	Development team receives a critical bug report.
Output/Result	Defect is fixed and deployed within FixTime.
Covers	MAS-MNT-03 and verifies timely corrective maintenance of critical defects.
How test will be performed	Historical defect reports, GitHub issues, pull requests, testing records, and deployment timestamps will be reviewed. The elapsed time between defect reporting and deployment will be measured.

test-MAS-MNT-04	Third-Party Integration Maintenance
Type	Static Review, Manual
Initial State	A supported third-party fitness provider changes its API.
Input/Condition	Existing integration becomes incompatible due to external API changes.
Output/Result	A compatibility update is deployed within FixTime of identifying the issue.
Covers	MAS-MNT-04 and verifies maintainability of third-party integrations and updates.
How test will be performed	An integration-breaking change will be simulated or identified from a real provider update. Development records and deployment dates will be reviewed to confirm compliance with the five-day requirement.

Supportability Covers MAS-SUP-01.

test-MAS-SUP-01-01	In-App Support Request Submission
Type	Dynamic, Manual
Initial State	User is logged into the application and has a support request they would like to raise.
Input/Condition	User opens the support feature and submits a support request.
Output/Result	Support request is successfully submitted and recorded by the system.
Covers	MAS-SUP-01 and verifies the ability for users to request assistance from within the application.
How test will be performed	The tester will navigate to the support section, submit a support request, and verify the request is recorded successfully and visible to support personnel.

test-MAS-SUP-01-02	Support Response Time Verification
Type	Static Review, Manual
Initial State	At least one submitted unresolved support request exists.
Input/Condition	Review support logs and response records.
Output/Result	A response regarding resolution plans and timelines is provided within SupportResponseTime.
Covers	MAS-SUP-01 and verifies compliance with required support response times.
How test will be performed	A submitted unresolved support request will be sampled and reviewed. The tester will submit the response to the user verifying that it is sent immediately and verifying that the response details can be found by the user. Timestamps will be compared to response records to verify compliance within SupportResponseTime.

Adaptability Covers MAS-ADPT-01 to MAS-ADPT-03.

test-MAS-ADPT-01	New Fitness Provider Integration
Type	Dynamic, Manual
Initial State	A new third party fitness provider has been approved to integrate with the Flexin' application.
Input/Condition	User attempts to connect the new provider to their account to track activity data.
Output/Result	The provider can be connected using the same workflow as existing providers.
Covers	MAS-ADPT-01 and verifies adaptability to future potential fitness providers.
How test will be performed	The tester will connect both an existing provider and the newly added provider. The setup process will be compared to verify workflow consistency.

test-MAS-ADPT-02	Mobile Platform Portability
Type	Dynamic, Manual
Initial State	User has an existing account and they wish to access the application on an iOS mobile device.
Input/Condition	User signs in using an existing account from the iOS device.
Output/Result	Account information, activity history, and portfolio data are successfully retrieved without migration steps.
Covers	MAS-ADPT-02 and verifies future cross-platform portability.
How test will be performed	A compatible iOS client will authenticate using an existing account. The tester will verify successful data retrieval and consistency with Android data.

4.2.6 Security

These tests cover the Security Requirements section of the SRS (**?**), spanning access control, data and logic integrity, privacy, economic and security audit logging, and immunity against attacks.

Access Covers SEC-ACC-01 to SEC-ACC-05.

test-SEC-ACC-01	**Authenticated Access Token Validation**
Type	Dynamic, Automated
Initial State	Backend server running in test profile with test database; user account registered; valid access token issued with a 60-minute expiry window.
Input/Condition	Send requests to a protected system resource (such as fitness activities or portfolio summaries) under four conditions: (1) missing authentication credentials, (2) tampered token signature, (3) expired token (timestamp beyond the 60-minute lifetime), and (4) authentic, unexpired token.
Output/Result	Requests lacking credentials, with a tampered token, or with an expired token are rejected with HTTP 401 Unauthorized. The request with a valid token returns HTTP 200 OK with the requested user data.
How test will be performed	Automated integration tests dispatching requests across token states against protected system services.

test-SEC-ACC-02	**Role-Based Authorization and Cross-User Isolation**
Type	Dynamic, Automated
Initial State	Two standard test users (User A and User B) with distinct portfolios and data records, and one administrator user.
Input/Condition	(1) User A attempts to read User B's private portfolio. (2) User A attempts to execute a trade or modify records on User B's behalf. (3) User A attempts to access system-wide administrative audit logs. (4) Administrator requests access to system-wide administrative audit logs.
Output/Result	Cross-user reads, cross-user modifications, and standard-user administrative requests are rejected with HTTP 403 Forbidden with zero data modifications. The administrative request is accepted (HTTP 200 OK) returning the requested audit records.
How test will be performed	Automated security integration tests verifying role-based access control rules and tenant isolation across system boundaries.

test-SEC-ACC-03	**Secure Credential Storage and Secret Exclusion**
Type	Static and Dynamic, Automated
Initial State	Repository source commit under evaluation; mobile test environment.
Input/Condition	(1) Execute secret scanning on the complete version control repository history for API keys, private keys, and client secrets. (2) Store third-party fitness integration credentials via the client application and inspect local persistent device storage.
Output/Result	(1) Repository secret scan detects zero exposed credentials or keys. (2) Integration credentials are encrypted at rest using platform-managed secure key storage (e.g., AES-256 GCM); zero cleartext tokens exist on local disk.
How test will be performed	Automated repository secret scanning in CI and mobile client instrumentation tests inspecting device storage.

test-SEC-ACC-04	Password Strength, Lockout, and Multi-Factor Authentication
Type	Dynamic, Automated
Initial State	User authentication service active with test database; common password dictionary loaded in memory.
Input/Condition	(1) Submit account registration requests using short passwords (fewer than MinPasswordLength), common dictionary passwords, and a strong password (at least MinPasswordLength). (2) Submit LockoutAttempts incorrect password attempts for a user account, followed immediately by an attempt with the correct password within LockoutDuration. (3) Advance system time past LockoutDuration (or submit a password reset) and retry with valid credentials. (4) For an account with multi-factor authentication (MFA) enabled, submit valid credentials, followed by an invalid one-time password (OTP), and then a valid OTP.
Output/Result	Short and dictionary passwords return HTTP 400 Bad Request; strong passwords return HTTP 201 Created. The LockoutAttempts consecutive failed attempts trigger account lockout, rejecting subsequent login attempts (including the correct password) with HTTP 423 Locked (or HTTP 403 Forbidden). Login succeeds once the lockout duration elapses. MFA challenge rejects invalid OTPs (HTTP 401 Unauthorized) and grants session access only upon valid OTP submission.
How test will be performed	Automated integration tests verifying authentication policies, lockout timers, and MFA verification flows.

test-SEC-ACC-05	Session Termination and Device Unlinking
Type	Dynamic, Automated
Initial State	Active user session with valid access and refresh tokens; client device linked to user account.
Input/Condition	(1) Client issues a sign-out request with its registered device identifier. (2) Attempt subsequent resource requests using the revoked access token. (3) Attempt to submit activity data from the unlinked device. (4) Present a session token that has been inactive for longer than SessionTimeout.
Output/Result	Sign-out completes successfully (HTTP 200 OK) and unlinks the device. Requests presenting revoked tokens, data submissions from unlinked devices, and expired inactive sessions are rejected (HTTP 401 Unauthorized or HTTP 403 Forbidden).
How test will be performed	Automated integration tests verifying session invalidation, token revocation repositories, and device link states.

Integrity Covers SEC-INT-01 to SEC-INT-03.

test-SEC-INT-01	Activity Telemetry Plausibility and Deduplication
Type	Dynamic, Automated
Initial State	Authenticated user account with 0 virtual currency.
Input/Condition	Ingest four workout telemetry payloads: (1) sustained cadence of 280 steps/min (exceeding MaxCadence), (2) overlapping workout intervals from two integrations, (3) duplicate submission of an identical workout external sync ID, and (4) normal plausible workout (120 steps/min, no overlap).
Output/Result	(1) Implausible cadence payload marked `UNVERIFIED` with zero virtual currency awarded. (2) Overlapping interval flagged `UNVERIFIED`; only unique segments credited. (3) Duplicate payload rejected by idempotency key with zero duplicate currency minted. (4) Plausible workout verified and virtual currency awarded.
How test will be performed	Automated unit and integration tests executing domain validation rules on activity telemetry payloads.

test-SEC-INT-02	**Server-Side Authority on Valuations and Trades**
Type	Dynamic, Automated
Initial State	Authenticated user with virtual currency balance; server-authoritative asset price set (e.g., 50.00 FLX for `FLEX_CORP`).
Input/Condition	(1) Client submits a trade request specifying a client-dictated execution price (e.g., 0.01 FLX) and total cost. (2) Client submits a currency reward claim specifying an arbitrary client-dictated reward amount.
Output/Result	(1) The system rejects the trade request with HTTP 400 Bad Request, or ignores the client-supplied price and executes strictly at the server-authoritative valuation (50.00 FLX). (2) The system rejects client-dictated reward amounts and calculates currency rewards strictly from verified workout metrics.
How test will be performed	Automated integration tests verifying server-side pricing enforcement and reward calculation authority.

test-SEC-INT-03	Database Backup Integrity and Restoration Drill
Type	Dynamic Operational Verification and Static Configuration Review
Initial State	Staging database populated with seed dataset of users, portfolios, and audit records; automated backup cron job configured.
Input/Condition	(1) Inspect cloud backup policy scripts for 24-hour backup interval and 30-day retention lifecycle. (2) Trigger an automated backup snapshot, record SHA-256 data checksums, restore onto a clean containerized database instance, and measure restoration time.
Output/Result	Configuration confirms daily backups and 30-day automated retention. Database restores completely within target limit ($<$ 15 minutes in test environment), with record counts and checksums matching 100%.
How test will be performed	Automated staging script executing backup creation, containerized restoration, and data integrity verification.

Privacy Covers SEC-PRV-01 to SEC-PRV-03.

test-SEC-PRV-01	Encryption in Transit and at Rest
Type	Dynamic Security Verification and Database Inspection
Initial State	Staging backend deployed behind reverse proxy; database running with encrypted tablespace.
Input/Condition	(1) Execute network handshake scan against API endpoints testing plain HTTP and TLS protocols 1.0, 1.1, 1.2, and 1.3. (2) Insert sensitive biometric fields (heart rate, body weight) into database and inspect raw storage files on disk.
Output/Result	(1) Plain HTTP requests and TLS connections below version 1.3 are rejected; only TLS 1.3 handshakes succeed. (2) Direct inspection of database disk blocks reveals AES-256 ciphertext; sensitive biometric data is unreadable in raw form.
How test will be performed	Automated script combining TLS protocol scanning and raw disk inspection on test database volumes.

test-SEC-PRV-02	Private by Default and Leaderboard Opt-In Isolation
Type	Dynamic, Automated
Initial State	Newly registered user with logged workouts and simulated portfolio; default settings unchanged.
Input/Condition	(1) Query public user profile and public leaderboard for the new user. (2) User explicitly opts in to leaderboard sharing with a public alias. (3) User toggles leaderboard sharing off.
Output/Result	(1) By default, public profile returns 404 or minimal stub; user is omitted from public leaderboards. (2) Opted-in leaderboard displays only chosen alias and aggregate score; real name, email, and biometric metrics remain private. (3) Opting out removes the user from public leaderboards immediately.
How test will be performed	Automated API integration tests verifying endpoint responses across default, opt-in, and opt-out states.

test-SEC-PRV-03	User Account Deletion and PII Eradication
Type	Dynamic, Automated
Initial State	User account populated with profile, health logs, active session, and transactions in the financial audit ledger.
Input/Condition	User submits an account deletion request with credential confirmation. Inspect active application databases, attempt login with deleted credentials, and inspect the financial audit ledger.
Output/Result	All personal identifying information (name, email, device identifiers) and health logs are purged from the active system within DeletionWindow. Subsequent login attempts are rejected (HTTP 401 Unauthorized). The financial audit ledger preserves transactional continuity but replaces user identifiers with an irreversible pseudonym.
How test will be performed	Automated end-to-end integration test asserting database purge, login denial, and ledger anonymization.

Audit Covers SEC-AUD-01, SEC-AUD-02.

test-SEC-AUD-01	**Economic Event Ledger Immutability**
Type	Dynamic Database Privilege Verification
Initial State	Backend connected to database using standard application credentials; sample rows present in `economic_audit_ledger`.
Input/Condition	(1) Record simulated trade or reward via standard service API. (2) Attempt direct SQL `UPDATE` on an existing ledger record. (3) Attempt direct SQL `DELETE` on an existing ledger record.
Output/Result	Service API inserts new audit record with immutable timestamp. Direct SQL `UPDATE` and `DELETE` commands are denied by database role permissions with error 1142 (command denied); ledger records remain unmodified.
How test will be performed	Automated database integration test executing restricted SQL commands using application service credentials.

test-SEC-AUD-02	**Structured Security Event Logging**
Type	Dynamic, Automated
Initial State	Log appender active and configured for structured JSON output.
Input/Condition	Trigger security events: 3 failed login attempts, activity payload with tampered signature, rate limit exceeded (> 60 req/min), and unhandled server exception.
Output/Result	Logs capture structured JSON entries containing severity level (WARN/ERROR), ISO 8601 UTC timestamp, client IP, event type, and user ID (or anonymous). Zero sensitive credentials or plaintext passwords appear in log records.
How test will be performed	Automated integration test with in-memory log appender asserting on serialized JSON log structure and contents.

Immunity Covers SEC-IMM-01 to SEC-IMM-03.

test-SEC-IMM-01	**Injection Defense (SQLi, XSS, Command Injection)**
Type	Static SAST and Dynamic DAST Penetration Testing
Initial State	Backend service deployed in containerized test environment with database connectivity.
Input/Condition	(1) Run static analysis on system source code for unparameterized queries, unescaped reflection, and command execution. (2) Execute automated penetration scanning dispatching SQL injection, cross-site scripting (XSS), and command injection attack vectors across all external system interfaces.
Output/Result	Static analysis reports zero high-severity injection vulnerabilities. Penetration scanning reports zero exploitable injection vulnerabilities; malicious payloads are sanitized or rejected (HTTP 400/422).
How test will be performed	Automated security test pipeline combining static code analysis (SAST) and dynamic penetration scanning (DAST).

test-SEC-IMM-02	**API Rate Limiting Enforcement**
Type	Dynamic, Automated
Initial State	Rate limiting filter configured with threshold of 60 requests/minute per client IP.
Input/Condition	Client sends 60 consecutive requests (the RateLimit) within 30 seconds, sends 10 additional requests within the remainder of the 60-second window, waits 65 seconds, and sends a subsequent request.
Output/Result	Requests 1 to 60 process normally. Requests 61 to 70 return HTTP 429 Too Many Requests with a `Retry-After` header. The request following the cooldown period processes normally.
How test will be performed	Automated integration test using a multi-threaded HTTP client dispatching timed request bursts.

test-SEC-IMM-03	Client Environment Tampering and Root Detection
Type	Dynamic Verification
Initial State	Android test device or emulator with root privileges enabled (`su` binary present, test keys installed).
Input/Condition	(1) Deploy production release build onto rooted device and attempt app launch, fitness sync, and leaderboard access. (2) Deploy debug build onto rooted device and attempt same actions.
Output/Result	Production build detects untrusted environment, displays warning modal, and disables virtual currency minting and leaderboard features. Debug build runs without restrictions for testing.
How test will be performed	Automated Android UI test and manual release checklist verifying root detection library signals.

Security Review Walkthrough Static reviews are conducted once per milestone release cycle (Rev 0, Final Demo) by Daniel Franze-Da Silva (Scribe, UI/UX Design Lead), Arjun Karthik (GitHub Manager, CI/CD Lead), and an independent peer reviewer. The team reviews: (1) ISO/IEC 27001 Annex A mapping against system controls (COMP-STD-07), (2) least privilege scopes for third-party fitness APIs (SEC-ACC-03), (3) repository secret scan results, and (4) dependency vulnerability reports ensuring zero unresolved High/Critical CVEs.

4.2.7 Cultural

These tests cover the Cultural Requirements section of the SRS.

Cultural Covers CUL-01, CUL-02.

test-CUL-01-01	**Supports Canadian Languages**
Type	Dynamic, Manual
Initial State	User is logged into the application and the language setting is set to English.
Input/Condition	User navigates to Settings and changes the application language from English to French.
Output/Result	All user-facing screens, menus, labels, buttons, notifications, and educational content are correctly translated and displayed in French (Canada) with no untranslated text.
Covers	CUL-01. By passing this test, the application demonstrates support for Canadian French language localization.
How test will be performed	The tester will navigate through all major application screens after selecting French. The tester will inspect registration, login, dashboard, fitness goals, virtual currency, portfolio, educational content, settings, and notifications. The tester will verify that all visible text is translated into French and that no English text remains.

test-CUL-01-02	**Language Switching Persistence**
Type	Dynamic, Manual
Initial State	User1 has previously selected French as their preferred language. User2 has previously selected English as their preferred language.
Input/Condition	User1 closes and reopens the application. User1 logs out and logs into User2.
Output/Result	After User1 closes and reopens the application, all user-facing functionality remains displayed in French. After User1 logs out and logs into User2, all user-facing functionality is displayed in English to match the language settings set by User2.
Covers	CUL-01 by verifying that the individual user selected Canadian language remains active during future application use, regardless of account switching or application relaunching.
How test will be performed	User1 selects French, exits the application, relaunches Flexin', and verifies that all screens continue to appear in French without requiring the language to be selected again. User1 is logged out and logs into User2 verifying that all screens now appear in English to reflect User2's language settings.

test-CUL-02	Culturally Neutral Educational Content
Type	Static, Manual
Initial State	Educational content, tutorials, glossary entries, and sample investment scenarios have been implemented.
Input/Condition	Review all educational screens and learning material included in the application.
Output/Result	Educational content uses culturally neutral language and examples that can be understood regardless of cultural background, religion, ethnicity, or geographic region.
Covers	CUL-02 and verifies the requirement that educational content be inclusive and broadly applicable.
How test will be performed	Reviewers will inspect all educational lessons, glossary entries, onboarding examples, and sample market content. Reviewers will verify that no content assumes familiarity with a specific culture, religion, ethnicity, or geographic region and that explanations remain understandable to a general Canadian audience.

4.2.8 Compliance

These tests cover the Compliance Requirements section of the SRS.

Legal Covers COMP-LEG-01 to COMP-LEG-07.

test-COMP-LEG-01	**User Consent for Health Data Collection**
Type	Dynamic, Manual
Initial State	User account exists. Health data permissions have yet to be granted.
Input/Condition	User attempts to connect a supported fitness provider.
Output/Result	No fitness data is retrieved until the user explicitly completes the permission and consent process.
Covers	COMP-LEG-01 and verifies that explicit user consent is obtained before fitness data collection begins.
How test will be performed	The tester will attempt to connect a supported fitness provider while declining permission requests. The tester will verify that no activity information is synchronized. The tester will then grant permission and verify that activity synchronization succeeds.

test-COMP-LEG-04	**User Data Deletion Rights**
Type	Dynamic, Manual
Initial State	User account exists and contains fitness history and personal information.
Input/Condition	User selects the account deletion option from the application settings menu.
Output/Result	User account is successfully deleted and personal information is completely removed from any system components (i.e. databases, caches). The user is then forced to log out of the application.
Covers	COMP-LEG-04 and verifies compliance with user rights regarding disposal of personal information according to PIPEDA.
How test will be performed	The tester will call the account deletion action, confirm their intention to delete the account, verify that account access is removed, and verify that stored information is deleted from the database. The tester will then verify that they were logged out and attempt to login to the account that was just removed with corresponding credentials and be denied access.

test-COMP-LEG-05	AI-Generated Educational Content Disclosure
Type	Dynamic, Manual
Initial State	Educational content generated or assisted by an AI system is available.
Input/Condition	User views AI-assisted educational content.
Output/Result	A visible disclosure indicates that the content was generated or assisted by AI.
Covers	COMP-LEG-05 and verifies transparency requirements regarding AI-generated content.
How test will be performed	The tester will open educational content known to be AI-assisted and verify that the disclosure message is visible. The tester will also verify that application settings contain information regarding AI-assisted functionality.

test-COMP-LEG-06	Age Verification During Registration
Type	Dynamic, Manual
Initial State	User is on the account registration screen - register an new account.
Input/Condition	User enters a date of birth indicating an age below MinAge (18 years).
Output/Result	Account activation is denied and registration cannot continue until an age at or above MinAge is entered and verified by the system.
Covers	COMP-LEG-06 and verifies enforcement of legal age requirements.
How test will be performed	The tester will attempt to create an account using a birth date corresponding to an age below MinAge. The tester will verify that account creation fails. The test will be repeated using an age of MinAge and registration should succeed. The process will be repeated one more time using an age above MinAge and registration should succeed.

Standards Compliance Covers COMP-STD-01 to COMP-STD-07.

test-COMP-STD-01	**Google Play Store Compliance**
Type	Dynamic, Automatic
Initial State	Release build is ready for deployment.
Input/Condition	Submit release build through Google Play validation process.
Output/Result	Application successfully passes all mandatory Google Play checks.
Covers	COMP-STD-01 and verifies Android platform compliance.
How test will be performed	The release build will be uploaded to the Google Play Store testing environment and validation results will be reviewed for errors or policy violations.

test-COMP-STD-03	Development Standards Compliance Review
Type	Static Code Review, Automatic
Initial State	Source code, database schema definitions, and CI/CD configuration are available for review.
Input/Condition	Execute the project's automated validation pipeline with linters and review generated reports.
Output/Result	TypeScript code passes configured linting and formatting checks, Java code passes configured linting and formatting checks, and all database schema definitions and queries execute successfully against the supported MySQL version without warnings or errors.
Covers	COMP-STD-03, COMP-STD-04, and COMP-STD-05. Verifies compliance with TypeScript coding standards, Java coding standards, and database compatibility requirements.
How test will be performed	A pull request containing representative frontend, backend, and database changes will be submitted. The CI/CD pipeline will automatically execute ESLint/TypeScript validation, Java linting and formatting validation, and database schema/query verification scripts against the supported MySQL environment. The test passes only if all checks complete successfully with no errors or warnings preventing merge approval.

4.2.9 User Documentation and Training

These tests cover the User Documentation and Training Requirements section of the SRS (**?**), spanning in-app educational documentation, setup guides, help systems, and user training onboarding mechanisms.

User Documentation Covers DOC-01 to DOC-04.

test-DOC-01	Financial and Fitness Glossary Presence
Type	Static Inspection
Initial State	Application build running on a test device; glossary view accessible in-app.
Input/Condition	Open the in-app glossary and search for required educational terms (compound interest, inflation, ETFs, bond yields, and portfolio diversification).
Output/Result	Binary: Pass if the searchable glossary is present and contains entries for all required terms; Fail otherwise.
How test will be performed	Manual walkthrough inspection of the glossary view verifying term presence and search functionality.

test-DOC-02	Fitness Integration Guide Presence
Type	Static Inspection
Initial State	Application build running on a test device; integration settings or help section open.
Input/Condition	Navigate to the fitness integration configuration section and open the setup guide.
Output/Result	Binary: Pass if illustrated setup instructions with screenshots are present for all supported fitness providers (e.g., Garmin, Android Health Connect) and include troubleshooting guidance; Fail otherwise.
How test will be performed	Visual inspection confirming presence and completeness of illustrated setup and troubleshooting instructions.

test-DOC-03	Searchable Help Center Presence
Type	Dynamic, Semi-Automated
Initial State	Application open to the settings menu.
Input/Condition	Open the in-app help center from settings and enter search queries covering core topics (account management, activity sync, rewards, portfolio, simulated market, virtual currency, privacy).
Output/Result	Binary: Pass if the in-app help center opens without an external browser, contains articles covering all required topics, and returns keyword search results in under HelpSearchTime; Fail otherwise.
How test will be performed	Manual walkthrough and automated UI test timing keyword search queries and asserting topic article presence.

test-DOC-04	Digital Manual and Release Notes Presence
Type	Static Inspection
Initial State	Project repository on GitHub checked out at latest milestone commit.
Input/Condition	Inspect the documentation repository directory for the digital user manual and milestone release notes.
Output/Result	Binary: Pass if a self-contained user manual (PDF and Markdown formats) documenting architecture, navigation, data flows, and troubleshooting, along with milestone release notes, is present in the repository; Fail otherwise.
How test will be performed	Peer-review file audit by team members verifying presence and completeness of documentation artifacts in the repository.

Training Covers TRN-01 to TRN-03.

test-TRN-01	Interactive Onboarding Walkthrough Presence
Type	Dynamic, Manual
Initial State	Clean application state immediately after creating a new user account.
Input/Condition	Complete new account registration to trigger onboarding; inspect the four-step walkthrough (goals, rewards, simulated investing, compounding chart), test the "Skip Tour" option, and re-launch the walkthrough from settings.
Output/Result	Binary: Pass if the four-step walkthrough is present, takes under WalkthroughTime to complete, provides a visible "Skip Tour" button on every step, and is re-accessible from settings; Fail otherwise.
How test will be performed	Manual timed walkthrough verifying the presence of all four tour steps, skip buttons, and settings re-entry.

test-TRN-02	Progressive Feature Discovery Presence
Type	Dynamic, Automated
Initial State	Newly registered user account with zero trades and zero logged physical activity days.
Input/Condition	Navigate to trading and portfolio analytics screens as a novice user; then simulate logging three days of physical activity or executing an initial trade, and re-examine the screens.
Output/Result	Binary: Pass if advanced trading indicators and portfolio analytics remain hidden or collapsed initially and are revealed once the activity or trade threshold is met; Fail otherwise.
How test will be performed	Automated Jest test with React Native Testing Library asserting the presence and visibility of advanced features before and after threshold completion.

test-TRN-03	Demonstrator Quick-Start Kit Presence
Type	Dynamic Operational Verification
Initial State	Application development/staging build running; demo seeding script available.
Input/Condition	Execute the automated demo seeding script or trigger the demonstrator debug toggle.
Output/Result	Binary: Pass if the demonstrator guide is present and the seeding script loads SeedHistory of simulated activity and portfolio history within SeedLoadTime; Fail otherwise.
How test will be performed	Execution of the demonstrator script or debug toggle timing dataset initialization and verifying data display on trading and compounding charts.

4.3 Traceability Between Test Cases and Requirements

The table below maps each system test to the SRS requirement whose fit criterion it checks. The last column lists other requirements that the test also measures, such as load tests that rerun the speed checks. Requirement sections that do not have test cases yet are added to the table as their tests are written.

Table 12: Traceability between system tests and requirements

Test Case	SRS ID	Requirement	Also Supports
test-FUNC-01-01	FUNC-01	User Registration	—
test-FUNC-01-02	FUNC-01	User Registration	—
test-FUNC-02-01	FUNC-02	User Authentication	—
test-FUNC-02-02	FUNC-02	User Authentication	—
test-FUNC-02-03	FUNC-02	User Authentication	—

Test Case	SRS ID	Requirement	Also Supports
test-FUNC-02-04	FUNC-02	User Authentication	—
test-FUNC-03	FUNC-03	Third-Party Fitness Data Import	SC-04
test-FUNC-04-01	FUNC-04	Fitness Goal Creation	—
test-FUNC-04-02	FUNC-04	Fitness Goal Creation	—
test-FUNC-04-03	FUNC-04	Fitness Goal Creation	—
test-FUNC-05	FUNC-05	Fitness Goal Tracking	—
test-FUNC-06	FUNC-06	Reward Allocation	—
test-FUNC-07	FUNC-07	Virtual Currency Tracking	—
test-FUNC-08	FUNC-08	Portfolio Creation	—
test-FUNC-09	FUNC-09	Asset Purchase	—
test-FUNC-10	FUNC-10	Asset Sale	—
test-FUNC-11	FUNC-11	Transaction Recording	FUNC-09, FUNC-10
test-FUNC-12	FUNC-12	Portfolio Performance Calculation	—
test-FUNC-13	FUNC-13	Leaderboard Participation	—
test-FUNC-14	FUNC-14	Friend Management	—
test-FUNC-15	FUNC-15	Private Leaderboard Creation	—
test-FUNC-16	FUNC-16	Progress Reporting	FUNC-03, FUNC-04, FUNC-05, FUNC-06, FUNC-07, FUNC-12
test-LF-APP-01	LF-APP-01	Visual Hierarchy	—
test-LF-APP-02	LF-APP-02	Compounding Chart	—

Test Case	SRS ID	Requirement	Also Supports
test-LF-APP-03	LF-APP-03	Responsive Layout	UHR-ACC-04
test-LF-APP-04	LF-APP-04	Theme and Contrast	COMP-STD-02
test-LF-APP-05	LF-APP-05	Navigation	—
test-LF-STY-01	LF-STY-01	Approachable Tone	—
test-LF-STY-02	LF-STY-02	Virtual Currency Symbol	PR-SC-03, SC-05
test-LF-STY-03	LF-STY-03	Non-Intrusive Gamification	—
test-LF-STY-04	LF-STY-04	Component Consistency	—
test-UHR-EOU-01	UHR-EOU-01	First-Time User Workflow Evaluation	UHR-EOU-02, UHR-EOU-03, UHR-EOU-08
test-UHR-EOU-04	UHR-EOU-04	User Feedback and Error Recovery Evaluation	UHR-EOU-05
test-UHR-EOU-07	UHR-EOU-07	Consistency Review	—
test-UHR-EOU-09	UHR-EOU-09	Re-Learnability Study	—
test-UHR-PER-01	UHR-PER-01	Personalization and Internationalization Verification	UHR-PER-02, UHR-PER-03, UHR-PER-04, UHR-INT-01

Test Case	SRS ID	Requirement	Also Supports
test-UHR-LRN-01	UHR-LRN-01	Learning and Educational Effectiveness Evaluation	UHR-LRN-02, UHR-LRN-03, UHR-LRN-04
test-UHR-UND-01	UHR-UND-01	Understandability Review	UHR-UND-02, UHR-UND-03
test-UHR-POL-01	UHR-POL-01	Respectful Communication and Content Moderation Review	UHR-POL-02, UHR-POL-03, UHR-POL-04, UHR-POL-05
test-UHR-ACC-01	UHR-ACC-01	Accessibility Compliance Evaluation	UHR-ACC-02, UHR-ACC-03, UHR-ACC-04, UHR-ACC-05, UHR-ACC-06
test-PR-SL-01	PR-SL-01	App launch time	—
test-PR-SL-02	PR-SL-02	Response to input	—
test-PR-SL-03	PR-SL-03	Trade confirmation time	—
test-PR-SL-04	PR-SL-04	Activity sync time	—
test-PR-SC-01	PR-SC-01	Daily earning limits	—

Test Case	SRS ID	Requirement	Also Supports
test-PR-SC-02	PR-SC-02	No penalties for missed days	—
test-PR-SC-03	PR-SC-03	Simulated market notice	SC-05
test-PR-SC-04	PR-SC-04	Not financial advice statement	—
test-PR-SC-05	PR-SC-05	Exercise injury warning	—
test-PR-SC-06	PR-SC-06	Safe goal progression	—
test-PR-SC-07	PR-SC-07	Notification limits	—
test-PR-SC-08	PR-SC-08	Safe leaderboard rules	—
test-PR-PA-01	PR-PA-01	Exact money values	—
test-PR-PA-02	PR-PA-02	Consistent currency awards	—
test-PR-PA-03	PR-PA-03	No double counting	—
test-PR-PA-04	PR-PA-04	Portfolio value	—
test-PR-PA-05	PR-PA-05	Workout dates and late syncs	—
test-PR-PA-06	PR-PA-06	Realistic market simulation	—
test-PR-RFT-01	PR-RFT-01	Offline activity	—
test-PR-RFT-02	PR-RFT-02	Atomic writes	—
test-PR-RFT-03	PR-RFT-03	Failing data sources	—
test-PR-RFT-04	PR-RFT-04	Backend unavailable	—
test-PR-C-01	PR-C-01	Concurrent users	PR-SL-02, PR-SL-03
test-PR-C-02	PR-C-02	Two years of history	PR-SL-02
test-PR-SE-01	PR-SE-01	Adding a fitness source	—
test-PR-SE-02	PR-SE-02	Adding an asset type	—
test-PR-SE-03	PR-SE-03	Adding server capacity	PR-SL-02, PR-SL-03
test-PR-L-01	PR-L-01	Supported Android versions	SC-01
test-PR-L-02	PR-L-02	Keeping user history	—
test-PROD-01	PROD-01	Build and Quality Assurance	—

Test Case	SRS ID	Requirement	Also Supports
test-PROD-02	PROD-02	Public Store Installation	PROD-01, SC-06
test-MAS-MNT-01	MAS-MNT-01	Fitness Activity Addition Effort	—
test-MAS-MNT-02	MAS-MNT-02	Investment Asset Addition Effort	—
test-MAS-MNT-03	MAS-MNT-03	Corrective Maintenance Effort	—
test-MAS-MNT-04	MAS-MNT-04	Third-Party Integration Maintenance	—
test-MAS-SUP-01-01	MAS-SUP-01	In-App Support Request Submission	—
test-MAS-SUP-01-02	MAS-SUP-01	Support Response Time Verification	—
test-MAS-ADPT-01	MAS-ADPT-01	New Fitness Provider Integration	—
test-MAS-ADPT-02	MAS-ADPT-02	Mobile Platform Portability	—
test-SEC-ACC-01	SEC-ACC-01	Authenticated Access Token Validation	—
test-SEC-ACC-02	SEC-ACC-02	Role-Based Authorization and Cross-User Isolation	—
test-SEC-ACC-03	SEC-ACC-03	Secure Credential Storage and Secret Exclusion	—
test-SEC-ACC-04	SEC-ACC-04	Password Strength, Lockout, and Multi-Factor Authentication	—
test-SEC-ACC-05	SEC-ACC-05	Session Termination and Device Unlinking	—
test-SEC-INT-01	SEC-INT-01	Activity Telemetry Plausibility and Deduplication	—

Test Case	SRS ID	Requirement	Also Supports
test-SEC-INT-02	SEC-INT-02	Server-Side Authority on Valuations and Trades	—
test-SEC-INT-03	SEC-INT-03	Database Backup Integrity and Restoration Drill	—
test-SEC-PRV-01	SEC-PRV-01	Encryption in Transit and at Rest	—
test-SEC-PRV-02	SEC-PRV-02	Private by Default and Leaderboard Opt-In Isolation	—
test-SEC-PRV-03	SEC-PRV-03	User Account Deletion and PII Eradication	—
test-SEC-AUD-01	SEC-AUD-01	Economic Event Ledger Immutability	—
test-SEC-AUD-02	SEC-AUD-02	Structured Security Event Logging	—
test-SEC-IMM-01	SEC-IMM-01	Injection Defense	—
test-SEC-IMM-02	SEC-IMM-02	API Rate Limiting Enforcement	—
test-SEC-IMM-03	SEC-IMM-03	Client Environment Tampering and Root Detection	—
test-CUL-01-01	CUL-01	Supports Canadian Languages	—
test-CUL-01-02	CUL-01	Language Switching Persistence	—
test-CUL-02	CUL-02	Culturally Neutral Educational Content	—
test-COMP-LEG-01	COMP-LEG-01	User Consent for Health Data Collection	—
test-COMP-LEG-04	COMP-LEG-04	User Data Deletion Rights	—

Test Case	SRS ID	Requirement	Also Supports
test-COMP-LEG-05	COMP-LEG-05	AI-Generated Educational Content Disclosure	—
test-COMP-LEG-06	COMP-LEG-06	Age Verification During Registration	—
test-COMP-STD-01	COMP-STD-01	Google Play Store Compliance	—
test-COMP-STD-03	COMP-STD-03	Development Standards Compliance Review	COMP-STD-04, COMP-STD-05
test-DOC-01	DOC-01	Financial and Fitness Glossary Presence	—
test-DOC-02	DOC-02	Fitness Integration Guide Presence	—
test-DOC-03	DOC-03	Searchable Help Center Presence	—
test-DOC-04	DOC-04	Digital Manual and Release Notes Presence	—
test-TRN-01	TRN-01	Interactive Onboarding Walkthrough Presence	—
test-TRN-02	TRN-02	Progressive Feature Discovery Presence	—
test-TRN-03	TRN-03	Demonstrator Quick-Start Kit Presence	—

5 Unit Test Description

The unit tests depend on the module decomposition in the Module Interface Specification (MIS), which is not finished yet, so this section will be completed in the next revision of the plan. The subsections below give the approach the team will follow.

5.1 Unit Testing Scope

Every module the team writes will be unit tested. Third-party libraries and platforms, such as Spring Boot, React Native, Health Connect, and the Garmin Connect IQ SDK, are out of scope, as explained in the objectives in Section 2. Modules that handle virtual currency, trades, and portfolio values will be tested first, since errors there affect the correctness objectives of this plan the most.

5.2 Tests for Functional Requirements

For each module in the MIS, each access program will get at least one test of normal behaviour, plus tests for its edge cases and exceptions. Backend modules will be tested with JUnit 5 and mobile app modules with Jest, as described in Section 3.6. Instead of listing every input and output here, this section will point to the test code, so each test will have a name that says what it checks.

5.3 Tests for Nonfunctional Requirements

Where a single module has to meet a nonfunctional requirement on its own, such as the precision of currency calculations (PR-PA-01), its unit tests will check that requirement directly. Most nonfunctional requirements are already covered by the system tests in Section 4.

5.4 Traceability Between Test Cases and Modules

Once the MIS is complete, a table here will map each unit test to the module it covers, so the team can show that every module has been tested.

6 Appendix

This appendix gives the values of the symbolic constants used in the tests and the questions for the usability survey.

6.1 Symbolic Parameters

The test cases use symbolic constants instead of literal values, so a value only has to change in one place. Table 13 gives the value of each constant, where the value comes from, and the tests that use it. Most values come from the fit criteria of the SRS requirements (**?**). The second group of constants is defined in the SRS symbolic constants table, and this plan uses the same names and values. Values marked “initial” may change after user testing; if they do, the SRS table is updated first and this table is updated to match.

Table 13: Symbolic constants

Constant	Value	Source	Used in
Constants defined in this plan			
UsabilityPassRate	80% of participants	LF-APP-01, LF-STY-01, UHR-LRN-01	test-LF-APP-01, test-LF-STY-01, test-UHR-LRN-01
MinCohortSize	10 participants	Team decision	test-LF-APP-01
LocateTime	5 seconds	LF-APP-01	test-LF-APP-01
ChartReadTime	5 seconds	LF-APP-02	test-LF-APP-02
TestFontScale	130% system font scaling	LF-APP-03	test-LF-APP-03
MaxAnimationTime	1.5 seconds	LF-STY-03	test-LF-STY-03
DismissTime	200 ms	Team decision (LF-STY-03)	test-LF-STY-03
LaunchTrials	20 cold starts	PR-SL-01	test-PR-SL-01
LaunchTime	3 seconds	PR-SL-01	test-PR-SL-01
LatencyPassRate	95% of measurements	PR-SL-01 to PR-SL-03	test-PR-SL-01 to test-PR-SL-03
InputResponseTime	1 second	PR-SL-02	test-PR-SL-02
TradeConfirmTime	2 seconds	PR-SL-03	test-PR-SL-03
SyncTime	10 seconds	PR-SL-04	test-PR-SL-04
SyncTrials	10 repetitions	Team decision	test-PR-SL-04
HelpSearchTime	200 ms	DOC-03	test-DOC-03

Constant	Value	Source	Used in
WalkthroughTime	90 seconds	TRN-01	test-TRN-01
SeedHistory	30 days	TRN-03	test-TRN-03
SeedLoadTime	10 seconds	TRN-03	test-TRN-03
OnboardingTime	5 minutes	UHR-EOU-01	test-UHR-EOU-01
InactivityPeriod	6 months	UHR-EOU-09	test-UHR-EOU-09
RelearnTime	5 minutes	UHR-EOU-09	test-UHR-EOU-09
FeatureAddTime	2 months	MAS-MNT-01, MAS-MNT-02	test-MAS-MNT-01, test-MAS-MNT-02
FixTime	5 business days	MAS-MNT-03, MAS-MNT-04	test-MAS-MNT-03, test-MAS-MNT-04
SupportResponseTime	2 business days	MAS-SUP-01	test-MAS-SUP-01-02
CoverageTarget	80% code coverage	Team decision	Section 3.6
UsabilityTestPeriod	2 weeks	Team decision	Section 3.7
Constants from the SRS			
HealthyDailyActivity	10,000 steps or 60 minutes of recorded workouts (initial)	SRS	test-PR-SC-01
DailyEarnCap	The currency earned for 1.5 × HealthyDailyActivity (initial)	SRS	test-PR-SC-01
MaxGoalIncrease	10% per week (initial)	SRS	test-PR-SC-06
MissedGoalLimit	3 consecutive goal periods (initial)	SRS	test-PR-SC-06

Constant	Value	Source	Used in
MaxDailyNotifications	3 per day (initial)	SRS	test-PR-SC-07
OfflineSyncWindow	7 days	SRS	test-PR-RFT-01
SessionTimeout	30 days without use	SRS	test-SEC-ACC-05
MinPasswordLength	8 characters	SRS	test-SEC-ACC-04
LockoutAttempts	5 consecutive failures	SRS	test-SEC-ACC-04
LockoutDuration	15 minutes	SRS	test-SEC-ACC-04
MaxCadence	250 steps per minute, sustained	SRS	test-SEC-INT-01
RateLimit	60 requests per minute	SRS	test-SEC-IMM-02
DeletionWindow	24 hours	SRS	test-SEC-PRV-03

6.2 Usability Survey Questions

First Impressions & Onboarding

- How did you feel about the initial four-step walkthrough? Did you find it helpful, or did you just want to tap "Skip Tour"?
- Did the app feel overwhelming when you first opened it, or did the advanced trading features unlock at a comfortable pace after you logged some activity?
- How would you describe the tone of the app? Does the financial terminology feel intimidating, or supportive and approachable?

Navigation & Dashboard

- Without searching, can you immediately point to your current fitness progress and your overall portfolio value on the home screen?
- If you wanted to browse the market or read an educational article, which tabs at the bottom would you naturally tap?

Charts & Simulated Investing

- Looking at the Portfolio Compounding Chart, can you easily tell which line represents your workout earnings and which represents your simulated investment gains?
- When you come across financial terms like "expense ratio" or "dividend," did you notice the interactive tooltips? Were the definitions easy to grasp?
- Is it completely clear from the interface that FLX is a virtual currency and you aren't risking real money?

Gamification, Privacy & Support

- When you hit a milestone, does the reward animation feel fun, or does it block you from using the app? Did you know you could swipe to dismiss it?
- Do you understand exactly what parts of your profile are public on the leaderboard versus kept entirely private?
- If you needed help connecting your Garmin or Android Health Connect, where would you go in the app to find the setup guide?

7 Appendix — Generative AI Use Disclosure

7.1 AI Tools Used

- **Claude Opus 5.5**, through Claude Code
- [Peter: add the AI tool and version used for the reflection question 4 notes. —SS]

[Team members who used other AI tools while writing their sections should add them here and describe the use below. —SS]

7.2 How and Where Used

- **Reflection Drafting (Appendix — Reflection):** The team met and agreed on the answers to the reflection questions, and recorded them as notes. Claude was used to turn those notes into the written answers. For question 4, an AI tool was also used to help draft the notes on how each team member will learn their VnV skill. Each member chose their own skill and learning approach.
- **Front Matter and Symbolic Constants (Section 1, List of Tables, Appendix — Symbolic Parameters):** Claude was used to build the symbols, abbreviations, and acronyms table from the terms used in this document, to add captions to the document's tables for the list of tables, and to collect the values used in the system tests into the symbolic constants table. Constants that already exist in the SRS (**?**) reuse the SRS names and values.
- **Formatting Consistency (Document-wide):** Claude was used to find and fix formatting that differed between sections written by different team members, such as test case IDs that followed different naming patterns.

7.3 Verification of AI-Generated Content

All AI-assisted material was reviewed by the team before it was merged. Reflection answers were checked against the meeting notes, and each member checked that their own skill and learning approach was described correctly.

Every constant value was checked against the test that uses it and, where the constant comes from the SRS, against the SRS symbolic constants table. Every renamed test ID was checked against the traceability table and the requirement it covers. Changes were made through pull requests and reviewed by at least one other team member, and the team is responsible for the accuracy of the final document.

8 Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
   Our meetings worked well for the sections everyone had to contribute to, especially Section 2 (General Information). We talked through that section as a group and kept notes as we went. Noah couldn't make that meeting, but between the notes and a quick catch-up he understood what we had decided and wrote the section up without needing a second meeting.
   We also stuck to the internal deadlines we set, so everything was ready for review on Sunday morning instead of being finished at the last minute.
   Splitting up the work was easy because the test plan follows the SRS. Each person took the requirements they had worked on in the SRS and wrote the matching tests, which meant people were writing about parts of the system they already knew well.

2. What pain points did you experience during this deliverable, and how did you resolve them?
   Fewer people made it to meetings this time because of things outside our control, mostly illness. We handled this the same way that worked for Section 2: whoever attended took notes, and anyone who missed a

meeting caught up from those notes and checked in with the group if anything was unclear.
We also weren't consistent at first about how to write test cases. Some of us used tables and some didn't, and test IDs weren't formatted the same way. We agreed on one format: every test gets an ID tied to the requirement it covers (for example, FUNC-01-T-01), written in a table with the same fields (control, initial state, inputs, output, derivation, and how the test will be performed). Then we went back and updated the tests that didn't match.

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

   - **Peter:** Unit testing with JUnit.
   - **Daniel F.:** Integration testing.
   - **Daniel H.:** System testing.
   - **Arjun:** Static testing, and tracking the issues it finds.
   - **Noah:** Documenting test results properly and reporting on them.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

   - **Peter (JUnit):** Peter could learn JUnit from the official JUnit 5 documentation and tutorials, or from a video course or guided online tutorial that builds a test suite from scratch. He will use the official documentation, along with writing small practice tests against our own project's classes. The documentation is the most accurate and up-to-date source, and practicing on our own code means what he learns carries straight over to our verification work.
   - **Daniel F. (Integration testing):** Daniel F. could learn integration testing by studying the standard strategies (top-down,

bottom-up, and big-bang integration) in a textbook or course material, or by reading through the integration tests in open-source projects similar to ours. He will start with the reading on strategies and then apply it by mapping out how our modules interact. He picked this because choosing the right integration approach for our architecture has to happen before writing the tests, and it will show him where the riskiest module interfaces are.

- **Daniel H. (System testing):** Daniel H. could learn system testing by reading about requirements-based and black-box testing techniques, or by working through a hands-on tutorial that tests a complete application end to end. He will take the requirements-based approach and use our SRS to derive system-level test cases. Since the SRS is already written, tying the system tests to its requirements builds the skill and also checks that we cover everything we said we would deliver.
- **Arjun (Static testing and tracking):** Arjun could learn static testing by running static analysis tools such as linters on our codebase, or by learning formal review techniques like code inspections and walkthroughs. For tracking, he could use a defect tracker such as GitHub Issues or Jira. He will do both the static analysis tools and the tracking tool, starting with the tools. Running the tools gives him concrete experience right away, and logging what they find in an issue tracker as he goes lets him learn both skills together.
- **Noah (Documentation of results and reporting):** Noah could learn to document results by studying standard test report templates and examples, such as the IEEE 829 structure, or by asking the team and the course instructors for feedback on early drafts of his reports. He will do both, starting with the templates and then revising his drafts based on feedback. The templates give him a professional structure to follow, and the feedback makes sure the reports are clear and useful to the rest of the team.